Критерий нормальности Колмогорова-Смирнова

- **Анатолий М. Георгиевский, ИТМО**, 2026

- Генераторы PRNG: MWC64 - MWC128

- Критерий нормальности Колмогорова-Смирнова
  - Постановка задачи
  - Критерий согласия Колмогорова
  - Критерий согласия Кёйпера (KS_Kuiper)
    - Циклическая однородность
  - Критерий Крамера — Мизеса — Смирнова
  - Критерий Андерсона — Дарлинга
  - Хи-квадрат статистика Пирсона
  - Дополнительные критерии нормальности
  - Криптографические генераторы Hash-DRNG и CTR-DRNG
  - Пакеты тестирования ГПСЧ
  - Выборочное тестирование с использованием TestU01
    - Тесты нормальности и p-value
  - Дополнение внешними статистиками
    - Leading/Trailing Zeros
    - Difficulties
    - Trump Aces
    - Hamming-Weight Dependencies
  - Выбор скрамблера
  - Генерация последовательности псевдослучайных чисел с нормальным распределением
- Разработка теста многомерной однородности
  - Параметризация поверхности тора в 3D
  - Решётчатая структура ГПСЧ
    - Определения из теории решёток
    - Ключевые проблемы
    - Auto-regressive-модель в модульной арифметике
  - Универсальное Разложение матриц в форме Фробениуса
    - Ключевые концепции алгоритма

- Собственные значения и инвариантные подпространства
- Матрица-генератор в подпространстве Крылова
- Анализ скрамблеров через Rational Canonical Form

Постановка задачи

Мы исследуем отклонение распределения гистограммы эмпирического распределения (EDF) от теоретического или другого эмпирического распределения. Прежде всего нас интересует поиск отклонения эмпирической функции распределения(EDF) от нормального распределения, заданного функцией CDF — кумулятивной функцией распределения вероятности.

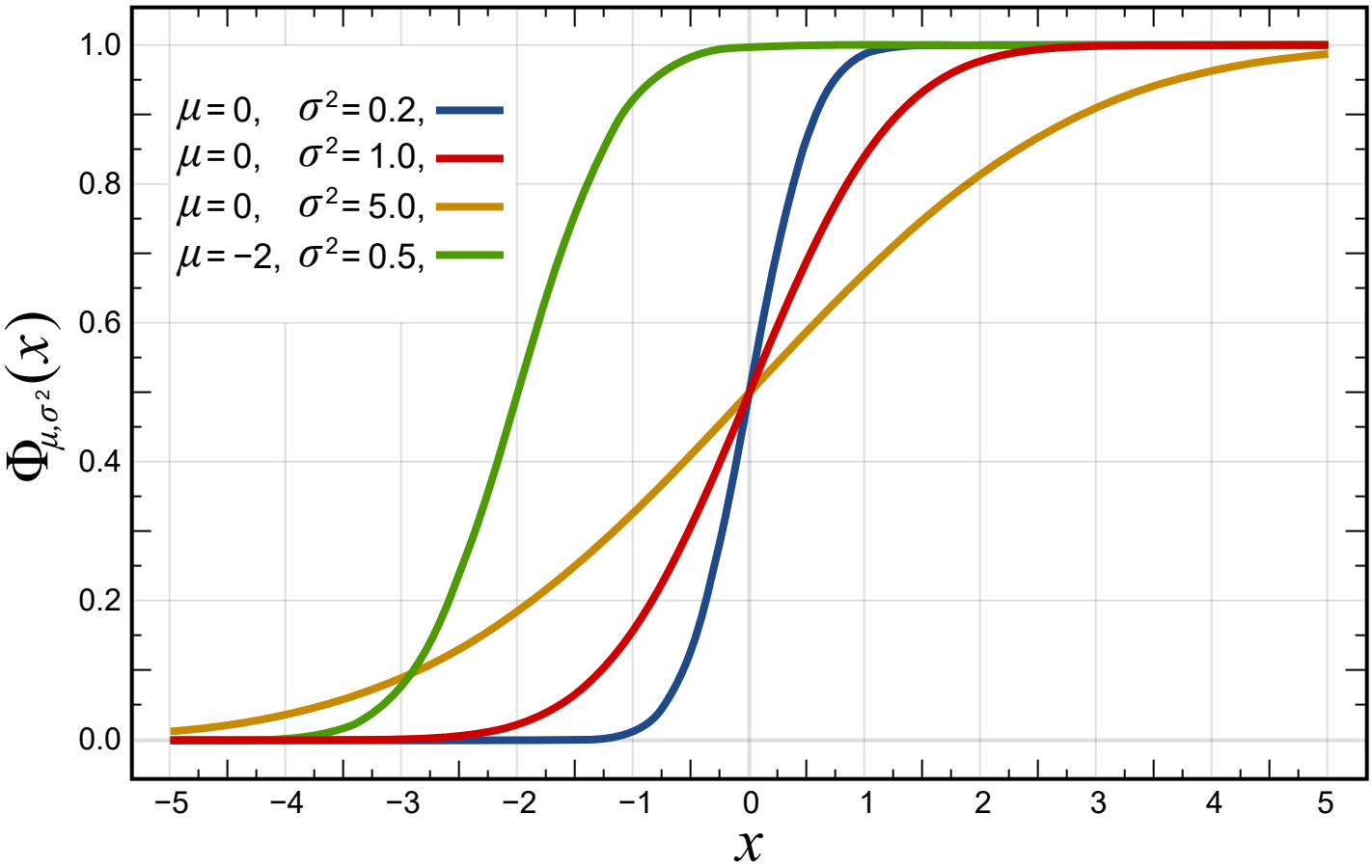
1. Мы хотим доказать или опровергнуть гипотезу нормального распределения положительных исходов H_0.
2. Мы исходим из предположения, что распределение с большим фиксированным временем накопления может оказаться ниже, чем распределение с динамическим управлением заданиями (окном).
3. При нормальном законе распределения мы планируем сформулировать критерий, который бы подходил для раннего выявления нерабочих режимов оборудования генерации случайных чисел.

Мы добавляем в ПО возможность анализа статистических данных и проверки гипотезы о нормальном распределении вероятности при данной выборке.

Возможные нерабочие состояния оборудования генерации ГПСЧ. Оборудование может эксплуатироваться вне рабочих режимов при повышении тактовой частоте, занижения напряжения питания и ограничения по времени окна накопления данных. Формально работоспособность можно проверить только путем загрузки тестовых заданий (seed/nonce), на которые известна тестовая последовательность отклика. Но этот метод не всегда доступен. Если оборудование частично неработоспособно, то при параллельной генерации часть диапазона не будет отсылать отклики, создавая паттерн в значениях. Паттерны могут проявляться в графическом, тензорном представлении. Статистика оперирует со случайными выборками, т.к. в нашем случае часть оборудования параллельной генерации может не отвечать в течение коротких или продолжительных не прогнозируемых интервалов времени. Некоторая неоднородность может возникать в процессе сбора данных из-за коллизий на линии.

Нормальный закон распределения плотности вероятности будет выражаться в терминах математического ожидания и дисперсии. Для математической оценки отклонения используются различные критерии, среди которых выделяется критерий Колмогорова-Смирнова, критерий Пирсона. Нас прежде всего интересуют методы основанные на вероятности распределения и

дающие строго математический критерий применимый к однородному или нормальному распределению плотности вероятности. Сам алгоритм генерации так же может иметь некоторую скрытую неоднородность, которая проявляется на специфичных значениях параметра (seed). Для исследования и моделирования поведения аппаратных генераторов мы используем простые 64/128 битные генераторы с большим периодом повтора, в частности MWC64/128 и Xoroshiro64/128. Обработка статистических данных с аппаратного генератора возможна с использованием файла выборочных значений из потоковых данных. Файл содержит параметры генератора (seed, timestamp) подобные тесту NIST `Longest Runs of Ones in a Block` с временными метками возникновения длинной последовательности единиц/нулей в блоке 64/256 бит. Длины прогонов единиц и ноликов (Runs) можно анализировать статистическими тестами.



CDF (Кумулятивная функция распределения для нормального распределения вероятности) — это функция, которая показывает вероятность того, что случайная величина X примет значение меньше или равное заданному числу x, т.е. $P(X \leq x)$. CDF является возрастающей функцией, начинается с 0 и достигает 1, преобразуя функцию плотности вероятности (PDF) в накопленное значение вероятности до точки x.

Положим, что X распределена *нормально*. Тогда CDF для X задается формулой:

$$F(t;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{t} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx.$$

Здесь параметр μ — это математическое ожидание распределения, а σ — его стандартное отклонение.

Положим, что X распределена *биномиально*. Тогда CDF для X задается формулой:

$$F(k;n,p) = \Pr(X \le k) = \sum_{i=0}^{\lfloor k \rfloor} \binom{n}{i} p^i (1-p)^{n-i}.$$

Здесь p — вероятность успеха, функция описывает дискретное распределение числа успехов в последовательности из n независимых экспериментов, а $\lfloor k \rfloor$ — целая часть k.

Рекомендации по анализу статистики представлены в ГОСТ Р 50.1.033-2001 и ГОСТ Р 50.1.037-2002:

- [Р 50.1.033-2001] РЕКОМЕНДАЦИИ ПО СТАНДАРТИЗАЦИИ. Прикладная статистика. Правила проверки согласия опытного распределения с теоретическим. Часть I. Критерии типа хи-квадрат.
- [Р 50.1.037-2002] РЕКОМЕНДАЦИИ ПО СТАНДАРТИЗАЦИИ. Прикладная статистика. Правила проверки согласия опытного распределения с теоретическим. Часть II. Непараметрические критерии (рассматривает критерий Колмогорова и критерий Смирнова).

См. также:

- Kolmogorov–Smirnov test
- Normality test

Критерий согласия Колмогорова

Применение критерия согласия А. Н. Колмогорова на практике осложняется следующими обстоятельствами. Для сопоставления функций эмпирического и теоретического распределений необходимо знать значения математического ожидания и дисперсии наблюдаемой случайной величины, которые могут быть неизвестны. Если возникает необходимость подбора параметров теоретического распределения по выборке, критерий не применим.

В 1933 г. А. Н. Колмогоров доказал важную теорему: если функция распределения $F(x)$ непрерывна, то при $n \to \infty$ вероятность максимального отклонения от теоретического распределения $\sup_x |F_n(x) - F(x)| < t/\sqrt{n}$ стремится к распределению Колмогорова. Операция *supremum* - выдает верхнюю границу отклонения D_n во всем диапазоне x.

Формулировка

Пусть $X_1, \ldots, X_n$ — выборка объёма n, порождённая случайной величиной, которая задаётся непрерывной функцией распределения $F(x)$. Пусть $F_n(x)$ — эмпирическая функция распределения. Тогда $\sqrt{n}\sup_{x\in\mathbb{R}}|F_n(x) - F(x)| \to K$ по распределению при $n \to \infty$, где K — случайная величина, имеющая распределение Колмогорова.

*Неформально говорят, что скорость сходимости эмпирической функции распределения к её теоретическому аналогу имеет порядок $1/\sqrt{n}$.

Обозначим нулевую гипотезу H_0 как гипотезу о том, что выборка подчиняется распределению $F(x) \in C^1(\mathbb{X})$. Тогда по теореме Колмогорова для введённой статистики справедливо:

$$\forall t > 0\colon \lim_{n\to\infty} P(\sqrt{n}D_n \leqslant t) = K(t) = \sum_{j=-\infty}^{+\infty}(-1)^j e^{-2j^2t^2} = \theta_4(e^{-2t^2}),$$

где $D_n = \sup_x |F_n(x) - F(x)|$.

Теорема Смирнова

Пусть $F_{1,n}(x)$, $F_{2,m}(x)$ — эмпирические функции распределения, построенные по независимым выборкам объёмом n и m случайной величины ξ. Тогда, если $F(x) \in C^1(\mathbb{X})$, то

$$\forall t > 0\colon \lim_{n,m\to\infty} P\left(\sqrt{\frac{nm}{n+m}}D_{n,m} \leqslant t\right) = K(t) = \sum_{j=-\infty}^{+\infty}(-1)^j e^{-2j^2t^2},$$

где $D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)|$.

Теорема Смирнова позволяет построить критерий для проверки двух выборок на однородность.

Критерий согласия, как его используют тестах запишется

$$\forall t > 0\colon \lim_{n,m\to\infty} P\left(\sqrt{\frac{nm}{n+m}}D_{n,m} > t\right) = 1 - K(t) = 2\sum_{j=1}^{+\infty}(-1)^{j-1}e^{-2j^2t^2},$$

Критерий согласия Кёйпера (KS_Kuiper)

> Изначально, я смотрел в код проекта `Dieharder` (набор тестов для проверки ГПСЧ), где применяется ряд вариантов теста KS, включая тест Кёйпера. В этой связи, следует упомянуть тест на нормальность.

Мы отдельно рассматриваем левые и правые отклонения, относительно нормального распределения $F(x,\theta)$, D_n^+ и D_n^-. В критерии Кёйпера используется статистика вида: $V_n = D_n^+ + D_n^-$.

Критерий согласия Кёйпера является вариантом критерия согласия Колмогорова и был предложен для проверки простых гипотез о принадлежности анализируемой выборки полностью известному закону, то есть для проверки гипотез вида $H_0 : F_n(x) = F(x,\theta)$ с известным вектором параметров теоретического закона.

Для однородного распределения используется сравнение с линейной кумулятивной функцией однородного распределения $F(x) = i/n$, нижняя и верхняя оценка:

$$D_n^+ = \max\left(\frac{i}{n} - F(x_i)\right),$$

$$D_n^- = \max\left(F(x_i) - \frac{i-1}{n}\right), \quad i = 1, ...n,$$

n — объём выборки, $x_1, x_2, ..., x_n$ — упорядоченные по возрастанию элементы выборки.

> Важно понимать, что тесты строятся с использованием последовательной выборки случайных чисел с применением генератора ГПСЧ с однородным распределение U(0,1). Тест предполагает сортировку массива данных. Выполнить сортировку для большого массива данных - длительная процедура. От критерия согласия Колмогорова-Смирнова в тесте остается только само распределение Колмогорова, которое рассчитывается асимптотически. В проекте Dieharder используются два метода расчета распределения Колмогорова, один из них - приближенный в тесте Кёйпера.

При справедливости простой проверяемой гипотезы статистика $\sqrt{n}V_n$ в пределе подчиняется[1] распределению:

$$G(v) = 1 - 2\sum_{n=1}^{\infty}(4n^2v^2 - 1)e^{-2n^2v^2}.$$

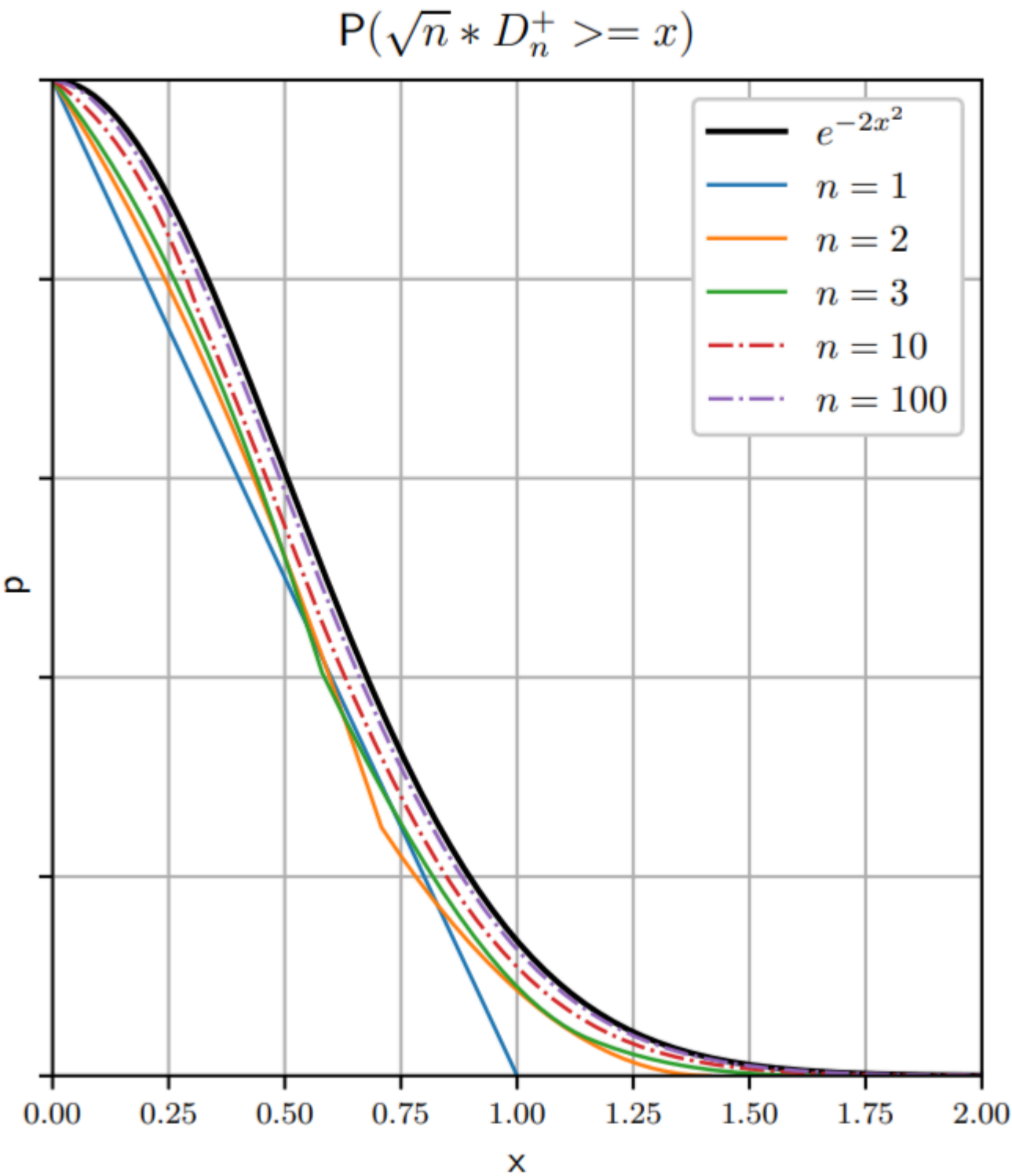
где n - число элементов выборки.

[1] Kuiper N. H. Tests concerning random points on a circle // Proc. Konikl. Nederl. Akad. Van Wettenschappen. 1960. Ser. A. V. 63. P. 38 — 47.

[1802.06966] Computing the Cumulative Distribution Function and Quantiles of the One-sided Kolmogorov-Smirnov Statistic

> Обзорная часть работы отсылает к односторонней границе Смирнова и точной формуле для вероятности (Smirnov-Birnbaum-Tingey).

$$\lim_{n\to\infty} \mathbb{P}(\sqrt{n}D_n^+ \le \epsilon) = \lim_{n\to\infty} \mathbb{P}(\sqrt{n}D_n^- \le \epsilon) = 1 - e^{-2\epsilon^2}$$



[503.11673] The Kolmogorov-Smirnov (KS) statistic revisited, 2025

> Мне нравится идея вернуться к математическим основам и вывести критерий для дискретной математики из мультиномиального (биномиального) распределения. Прежде всего четкий акцент - критерий согласия предназначен для сравнения EDF и CDF. Обсуждается идея использования оценки DKWM (Dvoretzky–Kiefer–Wolfowitz-Massart). Оценка дает коридор для эмпирической функции распределения.

В работе Представлена Теорема (Smirnov-Birnbaum-Tingey).
Точная формула вероятности превышения односторонней статистики имеет вид:

$$\mathbb{P}(D_n^- > \epsilon) = (1-\epsilon)^n + \epsilon \sum_{j=1}^{\lfloor n(1-\epsilon) \rfloor} \binom{n}{j} \left(\epsilon + \frac{j}{n}\right)^{j-1} \left(1 - \epsilon - \frac{j}{n}\right)^{n-j}.$$

В контексте теоремы Смирнова — Бирнбаума — Тингей и связанных с ней результатов по односторонним статистикам Колмогорова — Смирнова важную роль играет неравенство DKWM. Это неравенство даёт точную верхнюю оценку вероятности того, что максимальное отклонение эмпирической функции распределения $F_n(x)$ от истинной $F(x)$ превысит заданный уровень ϵ:

$$\mathbb{P}(D_n > \epsilon) \leq 2e^{-2\epsilon}$$

Полоса DKWM неасимптотическая (работает для любого конечного n), очень простая в вычислении и не требует таблиц.

Теорема Гнеденко — Рвачёвой — Феллера дополняет теорему Смирнова — Бирнбаума — Тингей (одновыборочный случай) и даёт распределение для одностороннего критерия Колмогорова — Смирнова с двумя выборками.

Следствие (асимптотика)
При $n = m \to \infty$

$$\mathbb{P}\big(D_{n,n}^+ \geq \frac{r}{\sqrt{n}}\big) \to e^{-r^2}.$$

> Это асимптотика нормального распределения вероятностей.

Циклическая однородность

> Критерий Кёйпера изначально разрабатывался для циклических (полярных) данных — углов, направлений, времени суток и т.п. Представляет собой критерий согласия (goodness-of-fit), который проверяет, соответствует ли выборка заданному распределению на окружности. Его ключевое преимущество — циклическая инвариантность (rotation invariance): результат не зависит от выбора начала отсчёта. Мы бы хотели представить критерий для однородного распределения U(0,1) по модулю простого числа или по модулю

> 2^w, рассмотреть частный случай модуля $A \cdot 2^s \pm 1$. Рассмотреть многомерную однородность.

Критерий Кёйпера для циклической однородности

Пусть даны углы $\theta_1, \ldots, \theta_n \in [0, 2\pi)$.

1. Преобразуем в единичный интервал: $U_i = \frac{\theta_i}{2\pi} \mod 1$.
2. Сортируем по возрастанию (гистограмма): $U_{(1)} \leq U_{(2)} \leq \cdots \leq U_{(n)}$.
3. Вычисляем односторонние отклонения:
   $D_n^+ = \max\limits_{i=1\ldots n} \left(\frac{i}{n} - U_{(i)}\right)$
   $D_n^- = \max\limits_{i=1\ldots n} \left(U_{(i)} - \frac{i-1}{n}\right)$
4. Статистика Кёйпера: $V_n = D_n^+ + D_n^-$

см. критерий Watson U^2

см. circular normal distribution

Нормальное циклическое распределение (вон Мизеса) von Mises distribution. The von Mises distribution is a special case of the von Mises–Fisher distribution on the N-dimensional sphere.

Wrapped binomial distribution на циклической группе $\mathbb{Z}/m\mathbb{Z}$ (дискретный круг с m точками) — это распределение, получаемое путём «обёртывания» (wrapping) обычного биномиального распределения Bin(n, p) по модулю m. Оно моделирует сумму n независимых шагов размера +1 (с вероятностью p) или 0 (с вероятностью 1-p), но с учётом периодичности по модулю m.

Функция вероятности для $r = 0, 1, \ldots, m-1$:

$$P(X = r) = \sum_{k=-\infty}^{\infty} \binom{n}{r + km} p^{r+km} (1-p)^{n-r-km}$$

(на практике суммируют по небольшому числу k, пока слагаемые не становятся пренебрежимо малыми).

Это один из простейших дискретных аналогов циклической нормальности (фон Мизеса), особенно при больших n и подходящем p.

Подробный разбор и характеристики (включая моменты, свойства, примеры применения) приведены в обзоре:

- "A Review of Wrapped Distributions for Circular Data" (2024), Mathematics, MDPI (раздел 3.1 Wrapped Binomial Distribution) (https://www.mdpi.com/2227-7390/12/16/2440)

Mathematics 2024, 12(16), 2440; https://doi.org/10.3390/math12162440
- Girija et al. (2014), "On Wrapped Binomial Model Characteristics", Mathematics and Statistics 2(7): 231-234, 2014 DOI: 10.13189/ms.2014.020701

Критерий Рэлея (Rayleigh test of uniformity) — статистический тест в циклической статистике.

Он проверяет нулевую гипотезу о том, что выборка углов (направлений) происходит из однородного распределения на окружности против альтернативы, что существует неравномерность в одном или нескольких направлениях.

Пусть есть n углов $\theta_1, \theta_2, \ldots, \theta_n \in [0, 2\pi)$.

Вычисляем векторную сумму:

$$C = \sum_{i=1}^{n} \cos\theta_i, \quad S = \sum_{i=1}^{n} \sin\theta_i$$

Статистика Рэлея:

$$z = nR^2 = \frac{C^2 + S^2}{n}$$

Статистика z имеет *асимптотическое* распределение χ^2 с двумя степенями свободы при $n \to \infty$.

Критерий Крамера — Мизеса — Смирнова

ω^2-статистика Мизеса. Cramér–von Mises test

Базовая идея - рассмотрение интеграла отклонения эмпирической кумулятивной функции от непрерывной кумулятивной функции нормального распределения.

$$\omega^2 = \int_{-\infty}^{\infty} [F_n(x) - F^*(x)]^2 \, \mathrm{d}F^*(x)$$

Заметим, что вместо интегрирования по dx предложено интегрирование по функции распределения dF(x), статистика не зависит от вида непрерывной функции распределения. Такое определение именуется как Smirnov's distribution-free statistic.

В статистических тестах почему-то вместо кумулятивной функции нормального распределения возникает сравнение с линейной функцией (кумулятивной функцией однородного распределения) и в тестах используется приближение

$$S = n\omega^2 = \frac{1}{12n} + \sum_{i=1}^{n} \left[F_n(x_i) - \frac{i - 0.5}{n} \right]^2 .$$

> Магия числа 12 не ясна. Очевидно эта поправка, для согласования критерия при малых n, связанная со способом дискретизации данных.

Критерий Андерсона — Дарлинга

Ω^2-статистика Мизеса. Anderson–Darling test

The Anderson–Darling and Cramér–von Mises statistics belong to the class of quadratic EDF statistics (tests based on the empirical distribution function).[2] If the hypothesized distribution is F, and empirical (sample) cumulative distribution function is F_n, then the quadratic EDF statistics measure the distance between F and F_n by

$$\Omega^2 = n \int_{-\infty}^{\infty} \left[F_n(x) - F(x) \right]^2 w(x)\, dF(x),$$

где n число элементов выборки, $w(x)$ - весовая функция.

В случае $w(x) = 1$, используется статистика Крмаера-Мизеса (Cramér–von Mises). Тест Андерсона-Дарлинга (Anderson–Darling) (1954) основан на квадратичной дистанции

$$\Omega^2 = n \int_{-\infty}^{\infty} \frac{\left[F_n(x) - F(x) \right]^2}{F(x)\,(1 - F(x))}\, dF(x),$$

с весовой функцией $w(x) = \left[F(x)\,(1 - F(x)) \right]^{-1}$. В таком виде тест может быть более чувствителен к отклонениям на краях распределения.

Формулу можно упростить используя асимптотику, для случая однородного распределения:

$$S_\Omega = -n - 2 \sum_{i=1}^{n} \left\{ \frac{2i - 1}{2n} \ln(F(x_i, \theta)) + \left(1 - \frac{2i - 1}{2n} \right) \ln(1 - F(x_i, \theta)) \right\},$$

где n — объём выборки,
$x_1, x_2, ..., x_n$ — упорядоченные по возрастанию элементы выборки.

- Anderson T. W., Darling D. A. Asymptotic theory of certain «goodness of fit» criteria based on stochastic processes // Ann. Math. Statist. — 1952. — V. 23. — P. 193—212.
- Anderson T. W., Darling D. A. A test of goodness of fit // J. Amer. Stist. Assoc., 1954. — V. 29. — P. 765—769.

> Хочется упростить статистику применительно к однородному распределению. Кумулятивная функция однородного распределения $F(x) = x$. Для анализа мы рассматриваем квадратичное отклонение от непрерывного распределения $F(x)$. И это должно напоминать среднеквадратичное отклонение. В идеале надо выражать нормировку отклонения в терминах функции распределения вероятности. И таким образом, мы приходим к критерию Пирсона. Следовало бы начать изложение с критерия Пирсона. Или вывести формулу критерия из формулы Байеса для полной вероятности. Далее рассмотреть асимптотику приводящую к использованию логарифмов в критериях нормальности, подобно тому как логарифмы возникают в машинном обучении.

Хи-квадрат статистика Пирсона

Данный раздел, следует начать с математической статистики.
[1] Б. А. СЕВАСТЬЯНОВ КУРС ТЕОРИИ ВЕРОЯТНОСТЕЙ И МАТЕМАТИЧЕСКОЙ СТАТИСТИКИ. ГЛАВА 14 Статистические критерии

> Я выделяю этот учебник, потому что он соответствует подготовке научной школы Колмогорова. Мне не нравится вводить критерии без обоснования. Большинство эмпирических методик тестирования ГПСЧ опираются на χ^2- статистику Пирсона.

В соответствии с заданным разбиением подсчитывают число
n_i выборочных значений, попавших в i-й интервал, и вероятности попадания в интервал
$P_i(\theta) = F(x_{(i)}, \theta) - F(x_{(i-1)}, \theta)$,
соответствующие теоретическому закону с функцией распределения $F(x, \theta)$.
Статистика критерия согласия χ^2 Пирсона определяется соотношением

$$\chi^2 = n \sum_{i=1}^{k} \frac{\left(n_i/n - P_i(\theta)\right)^2}{P_i(\theta)}.$$

Пример использования

Выдержка из документации TestU01:

```
void sstring_Run (unif01_Gen *gen, sstring_Res3 *res,
                  long N, long n, int r, int s);
```

Это вариант **теста на серии единиц** (run test), приспособленный для битовой строки.

В битовой строке длины n серии последовательных единиц (1) можно рассматривать как **зазоры** (gaps) между блоками последовательных нулей (0). Длины этих зазоров (или серий) являются независимыми **геометрически распределёнными** случайными величинами, увеличенными на 1; то есть вероятность того, что длина серии равна i, составляет 2^{-i} для $i = 1, 2, \ldots$. Естественно, то же самое верно, если поменять местами 0 и 1 и рассматривать серии нулей.

Тест строит битовую строку до тех пор, пока не получим ровно **n серий единиц** и **n серий нулей**, то есть всего **2n серий**.

Выбираем некоторое целое положительное число k.
Для j = 0 и j = 1 обозначим:

- $\chi_{j,i}$ — количество серий j-битов длины i, для i = 1, …, k−1
- $\chi_{j,k}$ — количество серий j-битов длины ≥ k

При справедливости нулевой гипотезы H_0 (битовая строка случайна) для каждого j вектор (X_{j,1}, …, X_{j,k−1}, X_{j,k}) имеет **мультиномиальное (многомерное биномиальное) распределение** с параметрами
(n, p_1, p_2, …, p_{k−1}, p_k),
где

- $P_i = 2^{-i}$ для $1 \le i < k$
- $P_k = 2^{-(k-1)}$, то есть p_k = p_{k-1}

Тогда, если величина $n \cdot P_k$ достаточно велика, статистика Хи-квадрат

$$\chi^2 = n \sum_{i=1}^{k} \frac{(n_i/n - P_i(\theta))^2}{P_i(\theta)(1 - P_i(\theta))}.$$

приближённо имеет **распределение хи-квадрат** с **k−1** степенями свободы.

Более того, величины χ_0^2 и χ_1^2 приближённо **независимы**, поэтому их сумма $\chi^2 = \chi_0^2 + \chi_1^2$

приближённо подчиняется распределению хи-квадрат с **2(k−1)** степенями свободы.
Функция расчета выполняется по предложенной статистике, для последовательностей нулей и

единиц.

```
X2 = 0.0;
for (j = runs_min; j <= runs_min; j++) {
        t = Count1[j] - nP[j];
    X2 += t * t / (nP[j] * (1.0 - P[j]));
}
```

Тут P_j - вероятность, которая дается как геометрическая прогрессия 1/2, 1/4, 1/8...; nP_j - произведение объема выборки n на вероятность. Подобным образом рассчитываются две статистики для ноликов и единиц, суммируются и выводится общий показатель, к которому применяются процентные точки p-value.

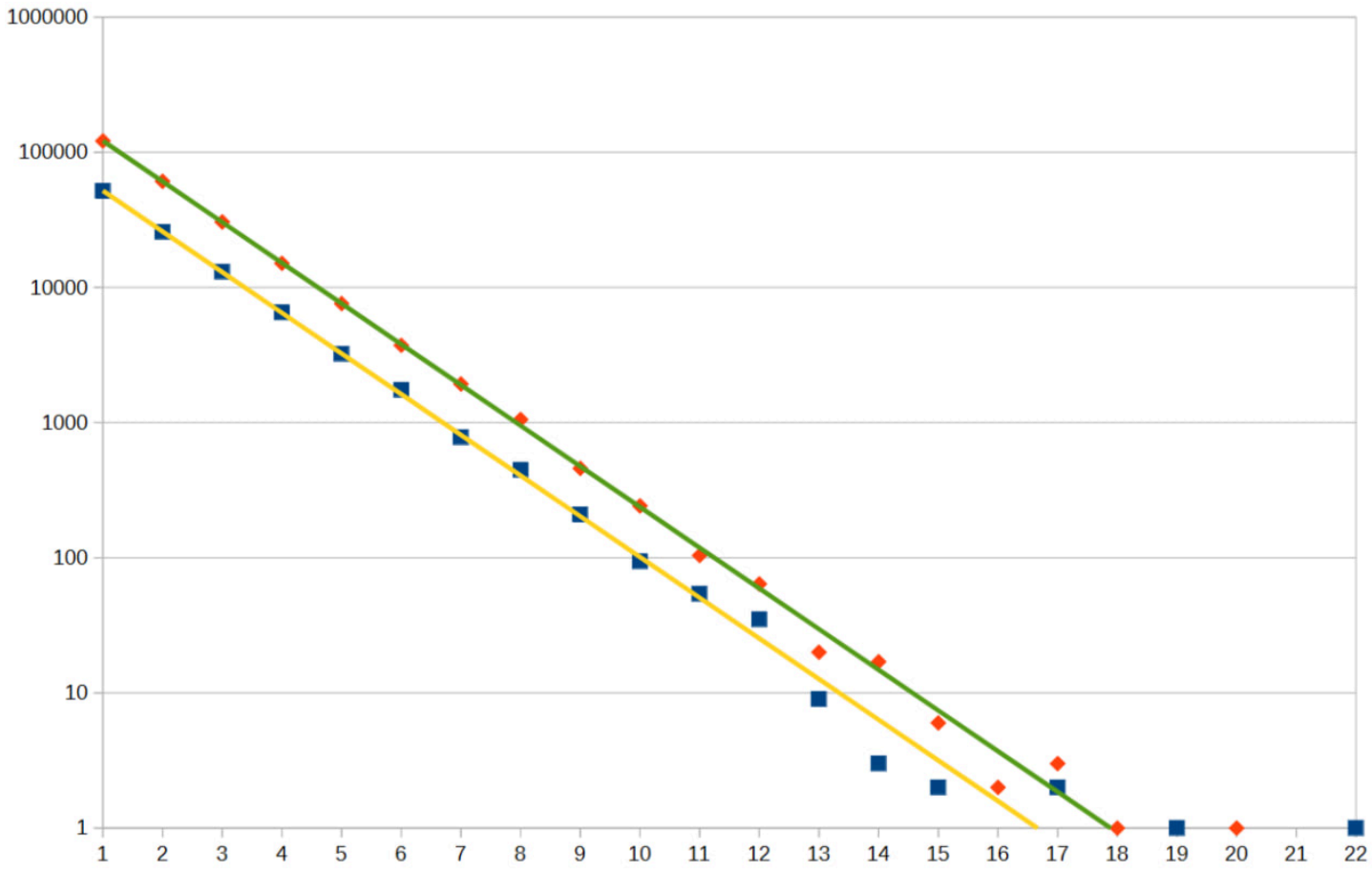


Рис. Пример распределения серий прогонов нулей, в логарифмическом масштабе для генератора типа Hash-DRNG. Представлены две независимые выборки и геометрическое распределение для каждой.

Заметим, что данные с малым числом nP вносят существенные шумы, и шум увеличивается с геометрической прогрессией при увеличении j обычно в расчет статистики берутся значения с $nP > 10$.

Оценка p-value выполняется по критерию Пирсона. В формуле метода Runs присутствует весовая функция $1/(1-P_i)$, которая проявляется только при малых степенях геометрической прогрессии P_i.

Дополнительные критерии нормальности

Помимо упомянутых, существуют другие тесты на нормальность:

- **Тест Андерсона-Дарлинга**: более чувствительная к отклонениям на краях распределения.
- **Тест Лиллиефорса**: Модификация Колмогорова-Смирнова для случаев, когда параметры распределения оцениваются по выборке.
- **Тест Жарке-Бера**: Основан на асимметрии и эксцессе, эффективен для больших выборок.
- **Критерий хи-квадрат (Pearson)**: Для проверки согласия с любым распределением, включая нормальное, требует дискретизации данных.
- **Тест Шапиро-Уилка**: статистический метод для проверки, насколько выборка данных соответствует нормальному (гауссову) распределению. Для небольших выборок.

Асимптоти́чески норма́льная оце́нка — в математической статистике оценка, распределение которой стремится к нормальному распределению при увеличении размера выборки. {применимо к результатам статистических тестов}

Асимптотически нормальная оценка (asymptotically normal estimator) — это одно из ключевых свойств точечных оценок в математической статистике, особенно важное при больших объёмах выборки (n → ∞).

Оценка $\hat{\theta}_n$, зависящая от выборки объёма n, параметра θ называется **асимптотически нормальной**, если выполняется сходимость по распределению:

$$\sqrt{n}\,(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}\big(0,\ \sigma^2(\theta)\big) \quad \text{при } n \to \infty$$

где $\sigma^2(\theta)$ — **асимптотическая дисперсия** (asymptotic variance) оценки,
$\xrightarrow{d}$ — сходимость по распределению (convergence in distribution, слабая сходимость).

Криптографические генераторы Hash-DRNG и CTR-DRNG

Криптографические генераторы упоминаются в связи с разработкой и применением методов тестирования однородности распределения. В работе мы используем раунды

криптографических функций и пермутации свойственные криптографическим функциям. Сравниваем методы тестирования криптографических генераторов случайных чисел и PRNG.

[NIST SP 800-90Ar1] Recommendation for Random Number Generation Using Deterministic Random Bit Generators.

Национальный стандарт NIST предлагает два варианта криптографических генераторов:

1. использовать в качестве основы хэш-функцию типа SHA256 или SHA3.
2. использовать блочный шифр AES в режиме CTR. или ChaCha20

Обозначим общую идею использования криптографической хэш-функции в режиме Hash-DRNG. Оговорка. В SHA3 функция KECCAK может использоваться в режиме генерации потока бит (SPONGE-SQUEEZE алгоритм - отжимание губки), см SHAKE. В SHA2 хэш-функция может применяться по-блочно. Для генерации используется хэш функция в режиме генерации потока бит,заданной длины `returned_bits = HashGen(bit_len, V)` . Аргументом является предыдущее состояние (`V`). Новое состояние `H = Hash(V); V = V+H+C+Seed` - это рандомизация счетчика `C` , состояния `V` и `Seed` , с использованием той же хэш-функции.

К генератором случайных чисел (PRNG) можно применить требования:

1. Функция перебора состояний без повторов с псевдослучайным выходом.
2. Функция рандомизации выходных значений для заданной длины (бит).

Принцип построения генератороа RNG рассмотрим на примере алгоритма MWC64x:

Состояние представлено, как пара чисел {x,c}: $S = (x + \beta c) \bmod P$, где $P = A\beta - 1$, $\beta = 2^s$.

1. Итерация выполняется на математической операции: $S = S\beta^{-1} \bmod P$.
2. Рандомизация (скрамблер) выходного значения: $x \oplus c$;

Так же из линейного генератора и скрамблера состоит метод Xoroshiro, который можно представить, как матричное (аффинное) преобразование в GF(2). Линейное преобразование делает генератор обратимым по шагам, см. reversible-RNG. Криптогорафические примитивы также рассматриваются, как обратимые, допускающие обратное преобразование раунда криптографической функции.

SPONGE-SQUEEZE — это обобщённая архитектура хэш-функции (XOF, сокр. eXtended Output Function), построенная по принципу *sponge construction*, но с явным разделением фаз и с акцентом на выравнивание состояния под машинное слово 32/64 бита.

Состояние состоит из двух сегментов (r + c) бит

- r — rate (видимая часть, для ввода/вывода)
- c — capacity (скрытая часть, обеспечивает безопасность)

r обычно выбирают кратным 32 или 64 битам для удобной работы на потоке 32/64-битных.

Этапы работы (SPONGE → SQUEEZE)

1. **Инициализация**
   `state = SEED` (иногда в capacity кладут IV или ключ)
2. **Padding** Форматирование сообщения
   `M → M || pad` (pad10*1 — самый распространённый, содержит длину)
3. **Absorbing (впитывание)** — для каждого блока r бит:
   `state[0…r-1] ⊕ блок` -- входное значение
   `state = f(state)` -- функция пермутации (генерация)
4. **Squeezing (выжимание)** — пока не получили заданную длину выхода:
   возвращаем `state[0…r-1]`
   `state = f(state)` -- генерация

Применительно к нашей теме, мы можем сконструировать хэш-фукнцию на оснвые генератора по принципу XOF (absorb-squeeze). Криптографическая функция пермутации должна обладать нелинейностью, чтобы затруднить или сделать вычислительно невозможным обратное преобразование без значния ключа. В модульной арифметике вычислительно сложной может быть функция возведения в степень или нахождение обратного числа и добавление раундовых констант.

Пакеты тестирования ГПСЧ

Существуют пакеты тестирования генераторов, которые позволяют сравнить в различных тестах. На практике используют тесты вроде NIST SP 800-22 (dieharder, TestU01) для проверки равномерности вывода бит генератора. По результатам теста генератор принимается нормальным или отвергается.

NIST, TestU01 и Dieharder

Сборка пакета тестирования TestU01 под GNU/Linux:

```
$ mkdir TestU01
$ cd TestU01
$ curl -OL http://simul.iro.umontreal.ca/testu01/TestU01.zip
$ unzip TestU01.zip
$ export basedir=`pwd`
$ cd TestU01-1.2.3/
$ ./configure --prefix="$basedir"
$ make -j 8
$ make -j 8 install
$ cd ../
$ mkdir lib-so
$ mv lib/*.so lib-so/.
$ gcc -std=c99 -Wall -O3 -o test-xoroshift.c test-xoroshift.c -Iinclude -Llib -ltestu01 -lprobd
```

Для примера приведу результаты тестирования Xoroshiro64* и MWC64x.

```
/* тест MWC64x с использованием библиотеки TestU01 */
#include "TestU01.h"
#define A1 0xFFFEB81BuLL
static uint64_t state[1]={~0};
static uint32_t mwc64x(void) {
    uint64_t x = state[0];
    state[0] = A1*(uint32_t)(x) + (x>>32);
    return x^(x>>32);
}
int main()
{
    // Create TestU01 PRNG object for our generator
    unif01_Gen* gen = unif01_CreateExternGenBits("MWC64x", mwc64x);
    // Run the tests.
    bbattery_SmallCrush(gen);
    // Clean up.
    unif01_DeleteExternGenBits(gen);
    return 0;
}
```

Заметим, что в метод MWC64 добавлена операция XOR: `x ^ (x>>32)` . Я бы назвал это словом "рандомизатор" или "скрамблер", эта операция позволяет улучшить прохождение некоторых тестов. С той же целью, в метод `Xoroshiro64` добавлено умножение результата итерации на большое простое число. Авторы метода xoroshiro предлагают несколько таких "скрамблеров", отчего алгоритмы отмечаются звездочками и плюсиками.

```
const uint64_t result_plus = s0 + s1;
const uint64_t result_plusplus = rotl(s0 + s1, R) + s0;
const uint64_t result_star = s0 * S;
const uint64_t result_starstar = rotl(s0 * S, R) * T;
```

Скрамблер - это линейное или нелинейное отображение, при заданной ширине слова, сохраняющее число состояний.

Для тестирования и сравнения генераторов предлагают методику *Hamming–Weight Dependencies* HWD. В публикации авторы ссылаются на ряд тестов "Hamming–Weight" из пакета TestU01, присутствующие в наборе Crush-тестов. Метод тестирования 64 битных генераторов, сводился к отдельному тестированию старшей и младшей части.

Для улучшения прохождения тестов авторы рекомендуют использовать старшие биты выхода генератора при переводе в вещественные числа.

Метод преобразования в вещественные числа $U[0,1)$, который реализует этот тезис выглядит следующим образом

```
float to_float(uint32_t x) {
    return (x>>8)*0x1.p-24f;
}
double to_double(uint64_t x) {
    return (x>>11)*0x1.p-53;
}
```

Альтернативный метод генерации double в диапзоне $U[0.5,1.0)$:

```
x = (x >> 12) | 0x3FF0000000000000ULL;
y = *((double *)&x);
```

Аналогично, в наших тестах при работе с MWC64 мы использовали преобразование

```
/* преобразование чисел в формат float32 дает распределение [0,1) */
static inline float u64_float(uint64_t x) {
        return ((uint32_t)((x>>32) ^ (x&0xFFFFFFFFU)) >> 8) * 0x1.0p-24f;
}
```

Пакет TestU01 предлагает *два* метода инициализации и использования генератора. Первый вариант мы протестировали - он на выходе дает значение 32 бита на итерацию. Второй вариант

работает в вещественных числах double и задает функцию генератора, как `double(*gen)()` :

```
double uniformX () {
    return (((uint64_t)mwc64_next()<<21))*0x1.0p-53;
}
int main() {
    unif01_Gen* gen = unif01_CreateExternGen01("MWC64x-U[0,1)", uniformX);
    bbattery_SmallCrush(gen);
    return 0;
}
```

По ссылке представлены результаты теста MWC64x в режиме генератора однородного распределения U(0,1). Для сравнения привожу результаты теста Xoroshiro64* при тех же условиях.

Для генерации однородного распределения float U(0,1) на GPU мы рекомендуем использовать генераторы MWC64x. Для генерации в формате double следует использовать генераторы 128 бит, такие как MWC128 и Xoroshiro128.

Hamming–Weight Dependencies test

[2108.13061] A New Test for Hamming–Weight Dependencies, 2022

Для тестирования мы приспособили тест [HWD]. Тест запускался без изменений с различнми генераторами MWC и Xoroshiro. см. prngs_hwd.c.

```
$ gcc -O3 -march=native -DHWD_BITS=64  -o hwd hwd.c
$ ./hwd
```

генератор MWC64x тест провалил. Все генераторы MWC128 и Xoroshiro64/128 тест проходят с числом итераций порядка 10^{12}.

Чтобы MWC64 проходил тест HWD в режмие 64-бита мы его вызываем дважды

```
#define MWC64_A0 0xFFFEB81BuLL
uint64_t mwc64x2_next() {
        uint64_t x = *s;
        x  = MWC64_A0*(uint32_t)(x) + (x>>32);
        *s = MWC64_A0*(uint32_t)(x) + (x>>32);
    return x^(x>>32);
}
```

В таком виде мы его применяем для генерации однородного и гауссова распределения. За одну итерацию MWC64 не полностью меняет состояние, итерация эквивалетна делению/сдвигу на 32 бита: $x \mapsto x \cdot 2^{-32} \bmod (A_0 \cdot 2^{32} - 1)$. Чтобы обновились все 64 бита, итерацию надо применять дважды.

Выборочное тестирование с использованием TestU01

Полный набор тестов из 140-150 методов может занимать существенно много времени. Следует выбрать методы наиболее подходящие для решения конкретной задачи. Тестовая задача - выбор параметров генерации MWC64/128 и выбор скрамблера выходных данных. При этом нас интересует использование генератора в режиме генерации однородного распределения U(0,1) в задачах моделирования физических процессов, таких как молекулярная динамика.

Скрамблеры - нелинейное преобразование. Источник идей - алгоритмы SHA1, SHA2, SHA3. Скрамблеры в алгоритмах Xoroshiro.

Скрамблеры - линейное преобразование.

- Грей код - обладает уникальным свойством в итерации меняется только один бит.
- Скрамблеры на циклических сдвигах и XOR.
- Скрамблеры на полиномиальной арифметике, используются нередуцируемые полиномы.
- Скрамблеры на аффинных преобразованиях.

Грей код в тестах применяется для выявления корреляций, а не как целевое решение. В генераторах ГПСЧ применять НЕ рекомендуется.

Зачем нам понадобились скрамблеры. Допустим мы иследуем некоторый генератор ГПСЧ, аргумент функции содержит seed и счетчик-итератор, который может увеличиваться линейно. Наш тест - проверить, как виляет применение линейного или нелинейного итератора на однородность распределения в различных статистиках и сравнить с линейным. Применяется статистика: AD, KS, χ^2.

Для выбора параметров генератора MWC64/128 и оценки эффективности линейных и нелинейных скрамблеров в задачах моделирования физических процессов (молекулярная динамика) можно использовать целевую подвыборку тестов из набора Crush пакета TestU01. Эта подвыборка фокусируется на выявлении артефактов, критичных для многомерной равномерности и линейных зависимостей, при этом остаётся выполнимой по времени (в отличие от полного набора тестов BigCrush).

см. Провальный пример теста линейности xoroshiro128+, то как можно использовать TestU01 с кастомным набором параметров. MWC64x при тех же настройках тест проходит.

Критерии отбора тестов при подборе парметров ГПСЧ и скрамблеров

- Многомерная равномерность — критична для молекулярной динамики, где последовательности используются для генерации координат/скоростей в 3D-пространстве и во времени.
- Чувствительность к линейным структурам — для оценки эффективности линейных скрамблеров (Грей-код, аффинные преобразования).
- Чувствительность к нелинейным зависимостям — для проверки скрамблеров, вдохновлённых хэш-функциями (SHA) и Xoroshiro.
  Битовая независимость — скрамблеры работают на уровне битов; важно проверить отсутствие корреляций.

Подвыборка тестов (из набора Crush TestU01)

Следующие 13 тестов обеспечивают баланс между глубиной анализа и временем выполнения:

Тест (имя в TestU01)	Тип проверки	Обоснование выбора
snpair_ClosePairs	Многомерная равномерность	Выявляет аномальные скопления точек в высоких размерностях — критично для распределения частиц в молекулярной динамике.
smarsa_BirthdaySpacings	Распределение расстояний в гиперкубе	Чувствителен к латтисной структуре линейных генераторов; эффективен против артефактов линейных скрамблеров.
scomp_LinearComp	Линейная сложность битовой последовательности	Прямо тестирует устойчивость к линейным зависимостям — ключевой тест для сравнения линейных и нелинейных скрамблеров.
scomp_MatrixRank	Ранг случайных бинарных матриц	Обнаруживает линейные артефакты в блоках битов; чувствителен к слабым линейным скрамблерам.

Тест (имя в TestU01)	Тип проверки	Обоснование выбора
sstring_HammingWeight2	Баланс 0/1 в последовательностях	с набором параметров из теста [HWD]
sstring_HammingIndep	Независимость весов Хэмминга	Выявляет корреляции в плотности единиц — критично для нелинейных преобразований (например, на основе модульной арифметики).
sstring_Run	Длины серий битов	Проверяет локальную независимость; чувствителен к артефактам Грей-кода (изменение одного бита).
sstring_LongestHeadRun	Максимальная серия единиц в блоке	выявляет экстремальные несбалансированные серии.
swalk_RandomWalk1/2	Случайные блуждания	Тестирует поведение в непрерывных пространствах; имитирует траектории частиц в динамике.
sknuth_MaximumTest	Распределение максимумов	Использует статистику AD/KS для оценки хвостов распределения — напрямую связано с критериями нормальности
sbit_TwoBitOverlap	Корреляции пар битов	Проверяет локальные нелинейные зависимости — важно для скрамблеров, имитирующих хэш-функции.
sspectral_Fourier1	DFT — периодические структуры	(эквивалент NIST DFT; выявляет циклы в 1D).
sspectral_Fourier3	многомерный сперктральный тест	Многомерный Fourier — периодичности в высших размерностях; усиливает выявление скрытых структур в многомерных данных. Рекомендуется для ГПСЧ в физическом моделировании, где

Тест (имя в TestU01)	Тип проверки	Обоснование выбора
		корреляции в координатах (x,y,z) могут искажать результаты.

Соответствие тестам NIST-800-22A

The NIST test suite The NIST (National Institute of Standards and Technology) of the U.S. federal government has proposed a statistical test suite for use in the evaluation of the randomness of bitstreams produced by cryptographic random number generators.

The NIST tests and the equivalent tests in TestU01 are:

1. The `Monobit` test. This corresponds to `sstring_HammingWeight2` with L = n.
2. The `Frequency` test within a Block. Corresponds to `sstring_HammingWeight2` .
3. The `Runs` test. Is implemented as `sstring_Run` .
4. The test for the `Longest Run of Ones` in a Block. Is implemented as the test `sstring_LongestHeadRun` .
5. The `Binary Matrix rank` test. Is implemented as `smarsa_MatrixRank` .
6. The `Discrete Fourier Transform` test. Is implemented as `sspectral_Fourier1` .
7. The Non-overlapping Template Matching Test,
8. The Overlapping Template Matching Test,
9. Maurer's "Universal Statistical" Test,
10. The Linear Complexity Test as `scomp_LinearComp` ,
11. The Serial Test,
12. The Approximate Entropy Test,
13. The Cumulative Sums (Cusums) Test,
14. The Random Excursions Test, and
15. The Random Excursions Variant Test.

[NIST.SP.800-22r1a] A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications, 2010

Тесты нормальности и p-value

Для реализации критериев нормальности тесты NIST предлагают использовать специальные статистические функции, среди которых - распределение Колмогорова, Гамма функция (`gamma`) и неполная Гамма функция (`igamc`), функция ошибки (`erfc`). NIST ссылается на библиотеку статистических функций CEPHES в составе Netlib.org. В стандарте Си присутствуют только `erfc` , `tgamma` , `lgamma` - Гамма функция и логарифм гамма функции. Реализацию неполной гамма функции мы копируем из пакета CEPHES double.

$$\Gamma(z,x) = \int_x^\infty t^{z-1}\, e^{-t}\, dt \quad \text{(верхняя неполная гамма-функция)}$$

Результат теста p-value, как правило, носит статистический характер и может существенно зависеть от SEED начальных условий. В этой связи предлагается многократно запускать тесты с разными начальными условиями SEED. Для оценики применяется "процентная ставка", задает уровень отсева результатов теста. Так для тестов Small_Crush установлен уровень `eps = 0.001`. Тест считается пройден, если $\epsilon < p < 1 - \epsilon$.

Некоторые тесты могут быть использованы для первичного анализа и требуют задания более жестких рамок. Например, граница для теста `BirthdaySpacings` очень хорошо отражает статистические свойства для первоначального отсева "слабых" параметров генератора. Для него Критерий выбора может быть установлен на уровне 0.1 - 0.25.

Результаты некоторых тестов оцениваются по статистике KS AD или χ^2. Для таких тестов можно применять усреднение и асимптотику от числа выборок. Установка более жестких границ p-value для таких тестов может быть ошибочная стратегия.

Подготовка данных:

- Для 64/128-битного генератора преобразуйте выход в double в $U(0,1)$
- При использовании 32-битных тестов используется разбиение 64-битного слова на два 32-битных. Старшие/младшие 32-битные слова тестируются незовисимо и вместе.
- Для выбора оптимального сочетания используется сравнительный анализ результатов теста для трех конфигураций
  i. Исходный MWC без скрамблера (базовый случай).
  ii. MWC + линейный скрамблер (например, Грей-код или аффинное преобразование).
  iii. MWC + нелинейный скрамблер (например, на основе свёртки с полиномами).

Сравните распределение p-value по тестам: удачный скрамблер должен «исправить» провалы базового генератора в `LinearComp`, `MatrixRank`, `BirthdaySpacings`.

Дополнение внешними статистиками

Для финальной верификации применяются собственые реализации критериев AD, KS, χ² к большим выборкам (10^8–10^9 итераций). Поверх теста применяется цикл прогона по начальным значениям SEED с использованием шага 2^{32} значений, внешний цикл усреднения по SEED - 512 итераций. Тест с разными начальными условиями SEED запускается паралелльно. На группу из N запусков выполняется пересчет среднего значения теста и максимального

отклонения $\sqrt{n} \cdot \max(D_n^+, D_n^-)$ с учетом статистики KS, AD или $\chi 2$. При увеличении числа измерений строится асимптотика для выбранной статистики.

Предложен ряд дополнительных тестов

Leading/Trailing Zeros

Методика расчета основана на статистике χ^2 и напоминает `Longest Runs of Ones in a Block`, но работает со словами 32 или 64 бита, в то время как тест [NIST-800-22A] предлагает 8 и 128 бит. Особенность теста - прогоны бит исследуются в старшей части числа с использованием операции `__builtin_clzll` - подсчет числа нулевых бит. Аналогично может быть выполнена операция подсчета числа нулевых бит от начала слова `__builtin_ctzll`.

Расчет вероятностей опирается на полную вероятность и геометрическую прогрессию 2^{-i}, где i - номер строки в таблице. Мы исходим из того, что сумма вероятностей приблизительно равняется удвоенному значнию в первой строке таблицы. Сумма частот в таблице будет давать число n.

- $nP_i = n \cdot 2^{-i}$, для $i = 1...M$

$$\chi^2 = \sum_{i=1}^{M} \frac{(x_i - nP_i)^2}{nP_i}$$

Условием суммирования ялвяется $nP_i \geq 100$.

Таблица. Значение χ^2 статистики для 2^{30} итераций, $\chi^2 = \chi_H^2 + \chi_L^2$

функция PRNG	текущее	среднее	пиковое
mwc64,A=FFFEF0E2	30.3	44.3	76.3
mwc64,A=FFFECC43	43.6	44.4	77.0
mwc64,A=FFFE7369	36.6	44.7	74.7
mwc64,A=FFFE60AF	39.1	43.7	76.0
mwc64,A=FFFE5BD5	32.6	44.8	77.7
mwc64,A=FFFE5A0A	43.2	45.1	83.8
mwc64,A=FFFE53E3	49.0	43.9	69.0
MWC128	44.9	43.8	

функция PRNG	текущее	среднее	пиковое
Xoroshiro128++	34.9	44.1	
sha256d	42.5	43.2	

Таблица. Значение χ^2 статистики для 2^{30} итераций CLZ

функция PRNG	текущее	среднее	пиковое
mwc64,A=FFFEFD4E	26.2	25.1	47.2
mwc64,A=FFFEF0E2	30.0	25.3	54.9
mwc64,A=FFFECC43	29.4	25.8	47.9
mwc64,A=FFFEA405	26.1	25.6	53.5
mwc64,A=FFFE7369	35.7	26.4	48.9
mwc64,A=FFFE60AF	39.7	26.7	52.4
mwc64,A=FFFE5BD5	25.8	26.9	42.9
mwc64,A=FFFE5A0A	26.0	27.4	51.4
mwc64,A=FFFE53E3	26.0	26.5	51.3
mwc64,A=FFFE03C4	23.5	26.3	50.7
MWC128	16.5	22.8	
xoroshiro128++	27.9	24.2	

В таблице представлен метод MWC64 с различным параметром и без использования скрамблера выходных значений. Для сравнения представлены MWC128, Xoroshiro128++ и Hash-DRNG на базе функции SHA256, функция SHA256 вызывается дважды. Все функции протестированы при одинаковых условиях, число итераций `2^30`, усреднение статистики (средняя колонка) выполнено по 512 циклам. Исоходный код теста CLZ test.

Difficulties

Данный тест заимствован из способа вычисления сложности шары в майнинге ... и может использоваться, как интегральный аналог теста `Leading Zeros`. Значения суммируются по

категориям. Значения, которые попадают в категорию N-бит нулей, суммируются со своей сложностью.

Категории могут выбираться, например, по времени накопления или по битовой сложности (числу нулевых бит) или по номеру генератора/вычислительного ядра, при параллельной генерации. Выбор категорий interleaving или с пропусками характерен для физических задач 3D. Следует рассматривать подвыборки: 1D, 2D, 3D ... 6D.

В каждой категории статистика (суммарная сложность) должна сохраняться. Так например при нормальном распределении суммарная сложность в каждом сегменте при параллельной генерации должна получится одинаковой.

Сложность вычисляется, как вещественное число по формуле:

```
double difficulty(uint64_t x) {
  return 1/((double)x + 0.5);
}
```

Вычисление сложности для чисел 128, 192 бит(порядок LE- little endian):

```
double difficulty128(const uint64_t *x)
{
  double value;
  value = x[1]
  value = x[0] + value * 0x1p64;;
  return 0x1p64/(value+0.5);
}


double difficulty192(const uint64_t *x)
{
  double value;
  value = x[2];
  value = x[1] + value * 0x1p64;
  value = x[0] + value * 0x1p64;
  return 0x1p128/(value+0.5);
}
```

Для разрядностей 256 - аналогично. Такой же алгоритм можно составить для вычисления сложности с точностью float и считать статистику в векторном виде с шагом 32 разряда. При

расчетах во FloatN необходимо следить, чтобы степень множителя помещалась в разрядность экспоненты вещественного числа.

Заметим, что может не хватать точности вычисления `double` для накопления результатов теста. В тесте мы использовали `_Float64x` или `long double` , - формат для накопления результата с числом итераций порядка 2^{32}.

Статистика многомерна и может применятся раздельно для параллельных генераторов! Результаты измерений в параллельных сегментах можно складывать.

Trump Aces

Тест проверяет точное число "козырных карт" в колоде. Тест предназначен для тестирования генераторотов с разрядностью 64 бита и выше. Колодой называется 2^{32} последовательных выборок с заданным начальным значением SEED. Например, если в колоде ровно 4 козырные карты, то получив в выборке три козырные карты за число выборок меньше 3/4, можно пересдавать колоду, такая стратегия окажется выгодной, особенно если "тузы" приходят парами. Если генератор состоит из слов по w-бит, то можно предположить, что каждая колода содержит свое фиксированное число "козырных карт" или наоборот, число козырных карт - случайная величина с нормальным распределением (проверяемая гипотеза H_0).

Для теста мы применяем паттерны состоящие из единиц и нулей:

- `pattern = 0xb000001` - "козырные тузы"/ степени двойки
- `pattern = (1u<<n)|1` - выделяет последовательность нулей между единицами
- `pattern = (1u<<n)-1` - последовательность n единиц

Заметим, что последовательность может не содержать степени двойки, но может содержать повторяющиеся паттерны. Повтор периода с установленной точностью считается провалом теста. Периоды должны иметь случайное нормальное распределение. Средний период не должен быть менее установленного порогового значения. Для генераторов MWC период решетки оценивается в (P-31/32) для 64 битной версии и (P-63/64) для версии 128 бит. Оценка периода повтора паттерна 64 бита: MWC128 > 2^{122}. Реально такие периоды мы не можем проверить перебором. Проверка выполняется для счетчика итераций от 2^{32} и выше по установленной маске 32 бита.

Для детектирования паттернов используется тест:

```
rotr32(r, __builtin_ctz(r))==pattern
```

или

```
rotl32(r, __builtin_clz(r))==pattern
```

в зависимости от способа выравнивания паттерна.

Тест применяется для генераторов с выходным скрамблером или без скрамблера.

- Исходный код теста для MWC32: mwc_gaps.c

Тестирование генераторов 128 бит и выше может использовать любое 32/64 битное слово из состояния генератора.

Для тестирования ARX-структур генераторов на базе блочных шифров и хэш-функций мы применяем обратную функцию, функцию обращения раунда к слову состояния, и берем слово состояния через определенное число раундов. Например для SHA это означает необходимость вычитания константы из выходного значения, чтобы перейти к состоянию генератора без функции скрамблера. см. reversible-RNG

Обобщение метода

> Рассмотреть вариант теста, с циклическим бинарным шаблоном, как сумму степеней двойки по модулю простого числа. Циклический сдвиг шаблона сопровождается модульной операцией сдвиг с редуцированием по модулю.

Hamming-Weight Dependencies

Тест заимствован у разработчиков Xoroshiro128.

[2108.13061] A New Test for Hamming–Weight Dependencies

Hamming Weight (вес Хэмминга) — это количество ненулевых символов в строке (в большинстве случаев — количество единиц в двоичной строке). В криптографии и обработке сигналов часто обозначается как $wt(x)$ или `popcount(x)` .

```
// Hamming weight for 64-bit integers
int hamming_weight64(uint64_t x) {
    return __builtin_popcountll(x);
}
// Hamming distance for 64-bit integers
int hamming_distance64(uint64_t x, uint64_t y) {
    return __builtin_popcountll(x ^ y);
}
```

Выбор скрамблера

[1805.01407] Scrambled Linear Pseudorandom Number Generators

Сначала нужно доказать, что метрика *сложность* предложенная в предыдущем параграфе достаточно точно отражает суть скрамблера, который не меняет *сложность* генератора, но улучшает однородность распределения и улучшает прохождение тестов. Для этого мы прогоняем через тест обычный счетчик `x = x+1` , который дает референсное значение (V_0) при полном прогоне 2^{32} итераций. Хороший скрамблер не меняет *сложность* распределения! Хороший скрамблер должен улучшать прохождение теста HammingWeight2 и HWD.

```
биты  сложность | частота
 0:        0.693| 2147483648
 1:        0.693| 1073741824
 2:        0.693| 536870912
 3:        0.693| 268435456
 4:        0.693| 134217728
 5:        0.693| 67108864
 6:        0.693| 33554432
 7:        0.693| 16777216
 8:        0.693| 8388608
 9:        0.693| 4194304
10:        0.693| 2097152
11:        0.693| 1048576
12:        0.693| 524288
13:        0.693| 262144
14:        0.693| 131072
15:        0.693| 65536
16:        0.693| 32768
17:        0.693| 16384
18:        0.693| 8192
19:        0.693| 4096
20:        0.693| 2048
21:        0.693| 1024
22:        0.693| 512
23:        0.693| 256
24:        0.693| 128
25:        0.693| 64
26:        0.693| 32
27:        0.693| 16
28:        0.693| 8
```

Далее мы выбираем *категории*, в нашем примере мы разбили диапазон 32 бита по 4 бита, по степеням. Мы сравниваем распределение сложности по категориям для нескольких скрамблеров и генераторов.

Реализация теста, см mwc_scrambler.c.

Подобный результат можно получить при переборе всего диапазона чисел 32 бита. Если преобразование однозначное, т.е. меняет порядок без уменьшения числа состояний, то результат суммирования сложностей по всему диапазону - константа. Т.е. любое линейное или нелинейное преобразование даст подобный результат. Так можно протестировать скрамблеры 32 битные.

Мы подобрали несколько линейных и нелинейных скрамблеров из числа операций, которые применяются при построении хэш-функций. В частности, мы рассмотрели скрамблеры предложенные для хэш функции на модульной арифметике [2023/1025]. Большой класс скрамблеров на базе LFSR и арифметике полиномиальной, мы не рассматриваем. Причина, по которой мы не уделили внимания этим скрамблерам - ориентация на GPU, в которых операция полиномиального умножения (умножения без переносов) не поддерживается. Некоторое подмножество LFSR с длинным периодом повтора может быть представлено через циклические сдвиги и операции XOR. В нашей работе этот класс попадает в линейные скрамблеры.
Подходящие скрамблеры типа LFSR следует искать среди нередуцируемых полиномов GF(2^m) для CRC-32, CRC-64.

- $x^{128} + x^7 + x^2 + x + 1$ и $x^{64} + x^4 + x^3 + x + 1$, используется в функции GHASH
- $x^{32} + x^{22} + x^2 + x^1 + 1$
- $x^{128} + x^{104} + x^{76} + x^{51} + x^{25} + x + 1$, используется для LFSR рандомизации в Linux-RNG

Среди скрамблеров можно рассматривать функции пермутации для большей разрядности, которые совмещают операции сложения циклического сдвига и XOR внутри слова и перестановку слов.

Скрамблеры применяем линейные и нелинейные:

Эффективные линейные скрамблеры:

$$x \mapsto x \oplus (x \lll i) \oplus (x \lll j)$$

Эффективные нелинейные скрамблеры:

$$x \mapsto x \oplus (\bar{x} \lll i) \odot (x \lll j)$$

Обращаем внимание, что подобные скрамблеры используются для пермутаций в множестве стандартов хэш-функций, включая SHA3 и SHA2.
Из стандарта SHA2 мы заимствовали 32-битные операции Sigma0, Sigma1, Sum0, Sum1 для изготовления тестовых скрамблеров:

$$x \mapsto (x \ggg 7) \oplus (x \ggg 18) \oplus (x \gg 3), \quad \text{- Sigma0 (32)} \tag{1}$$
$$x \mapsto (x \ggg 17) \oplus (x \ggg 19) \oplus (x \gg 10), \quad \text{- Sigma1 (32)} \tag{2}$$
$$x \mapsto (x \ggg 2) \oplus (x \ggg 13) \oplus (x \ggg 22), \quad \text{- Sum0 (32)} \tag{3}$$
$$x \mapsto (x \ggg 6) \oplus (x \ggg 11) \oplus (x \ggg 25), \quad \text{- Sum1 (32)} \tag{4}$$

64-битные операции Sigma0, Sigma1, Sum0, Sum1:

$$x \mapsto (x \ggg 1) \oplus (x \ggg 8) \oplus (x \gg 7), \qquad \text{- Sigma0 (64)} \tag{5}$$
$$x \mapsto (x \ggg 19) \oplus (x \ggg 61) \oplus (x \gg 6), \qquad \text{- Sigma1 (64)} \tag{6}$$
$$x \mapsto (x \ggg 28) \oplus (x \ggg 34) \oplus (x \ggg 39), \qquad \text{- Sum0 (64)} \tag{7}$$
$$x \mapsto (x \ggg 14) \oplus (x \ggg 18) \oplus (x \ggg 41), \qquad \text{- Sum1 (64)} \tag{8}$$

Заметим, скрамблеры построенные на битовой операции XOR, сдвигах и вращении можно представить в виде матрицы аффинного преобразования поле $GF(2^{64})$. Такие матрицы-Циркулянты обладают или могут обладать {уточнить} свойством Maximum Distance Separable (MDS).

В нашей работе использованы скрамблеры

$$x \mapsto x \oplus (x \lll 1), \qquad \text{- Gray} \tag{9}$$
$$x \mapsto x \oplus (x \lll 1) \oplus (x \lll 2) \tag{10}$$
$$x \mapsto x \oplus (\bar{x} \lll 1) \odot (x \lll 2) \tag{11}$$
$$x \mapsto x \oplus (\bar{x} \lll 1) \odot (x \lll 2) \odot (x \lll 3) \tag{12}$$

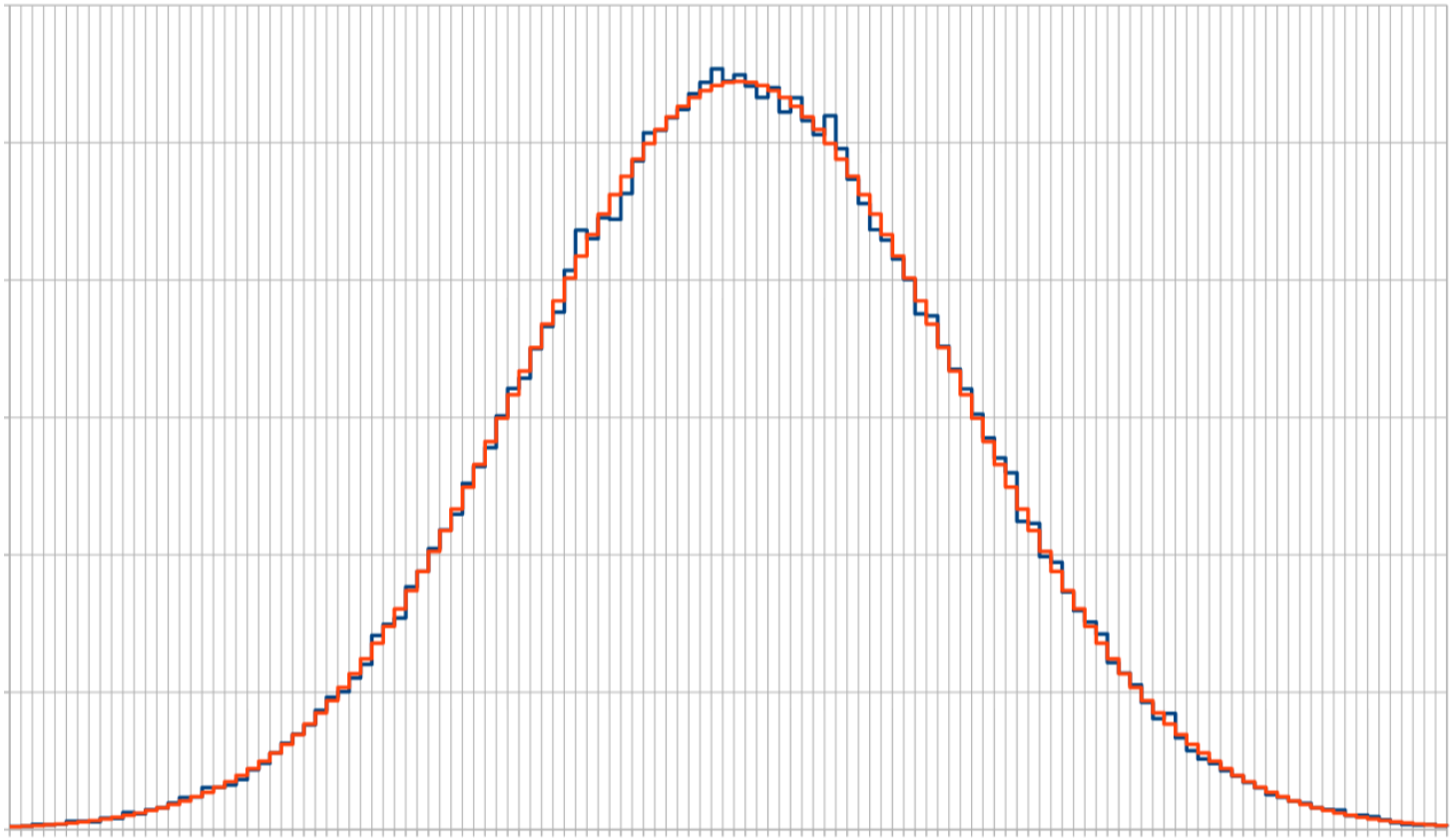
И скрамблеры перестановки байт, векторная операция `shuffle`

$$x \mapsto x \oplus (x \lll 32), \qquad \text{- XOR} \tag{13}$$
$$x \mapsto x \oplus (x \lll 8) \oplus (x \lll 16) \tag{14}$$
$$x \mapsto x \oplus (\bar{x} \lll 8) \odot (x \lll 16) \tag{15}$$
$$x \mapsto x \oplus (\bar{x} \lll 8) \odot (x \lll 16) \odot (x \lll 24) \tag{16}$$

Методы MWC с линейным скрамблером $x \oplus (x \lll 32)$ мы по традиции будем обозначать буквой x, например `MWC64x` .

Генерация последовательности псевдослучайных чисел с нормальным распределением

Тестовая задача - определить нормальность распределения полученного при использовании генераторов ГПСЧ. Отправная точка - построение гистограммы эмпирического распределения и сравнение с гистограммой нормального распределения при $n \to \infty$.



Гистограмма получена при накоплении результатов с использованием нормального распределения от генератора MWC64. Нормальное распределение получается из однородного с использованием преобразования Бокса-Мюллера.

Рекомендация: если мы тестируем нормальность после преобразования uniform → normal, Anderson–Darling выглядит предпочтительнее KS/Lilliefors. KS тест выделяет максимальное отклонение, в то время как AD считает суммарное отклонение с учетом весовой функции. KS не подходит, когда параметры μ, σ оцениваются по выборке. В нашем случае праметры распределения известны заранее.

Мы можем построить статистику χ^2 от двух графиков, поскольку речь идет о дискртеных значениях и построить зависимость предельного отклонения KS статистики от числа отсчетов. Если гипотеза верна, то будет наблюдаться сходимость статистики.

Разработка теста многомерной однородности

Мы хотим сформулировать алгоритм, который применяет принципы выявления гиперплоскостей в многомерном распределении. Пока что существует только концепция эксплуатирующая идею многомерного способа разложения статистики. В наших тестах мы использовали 2D разложения

статистики "салфетки", на которых могут проявляться сетки и полосы при подборе периода. Наш подход - графические тесты последовательностей. При наличии полос и периодических структур, может использоваться Фурье преобразование.

Общая идея графического теста - представление разложения на плоскости, на торе или в единичном кубе и поиск вектора нормали к гиперплоскости, в которой проявляется периодическая структура. Это можно представить методом поворота единичного кубика под углом, где проекция на плоскость имеет периодическую структуру.

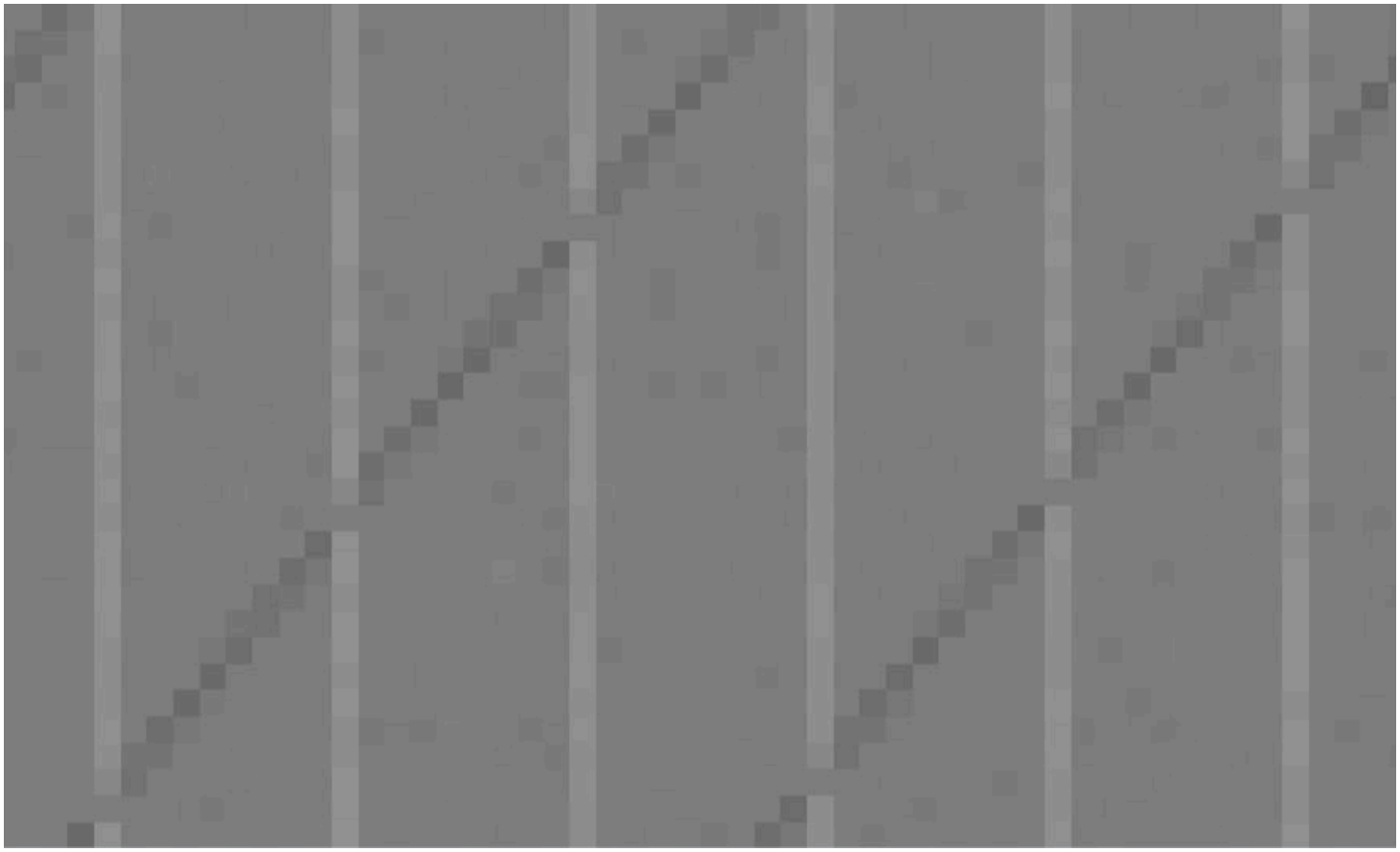


Рис. 1 Вывод статистики на плоскости, тест `MWC32` с параметром `A=0xFF00`. Выполнен подбор периода. Подбор выполнялся по критерию средне-квадратичного отклонения, при однородном распределении. Видно, что выделяются темные (D^-) и яркие полосы (D^+). Темные и яркие полосы имеют свой период повтора.

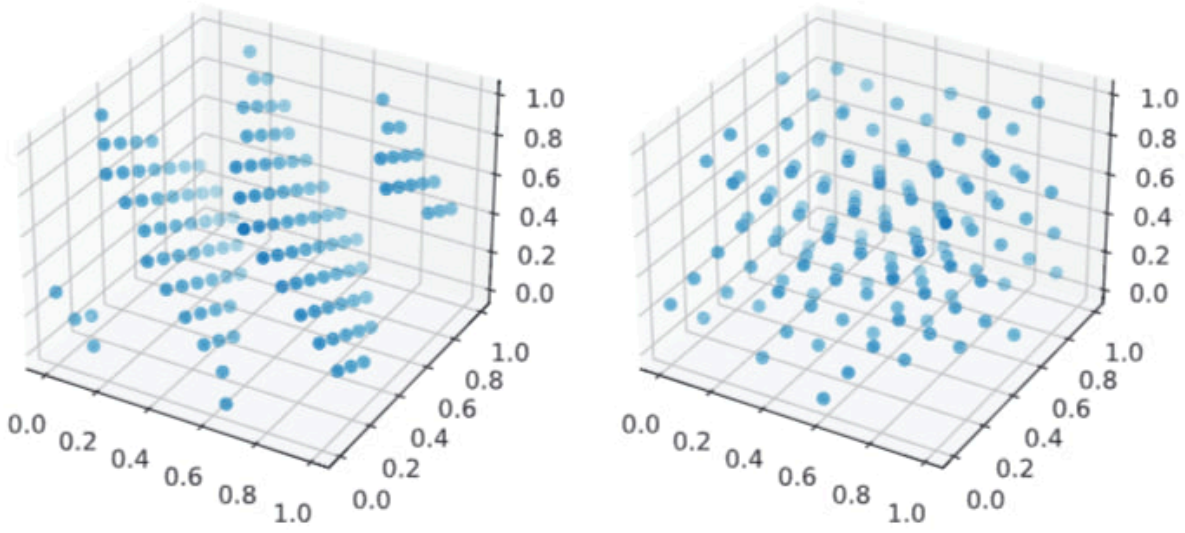
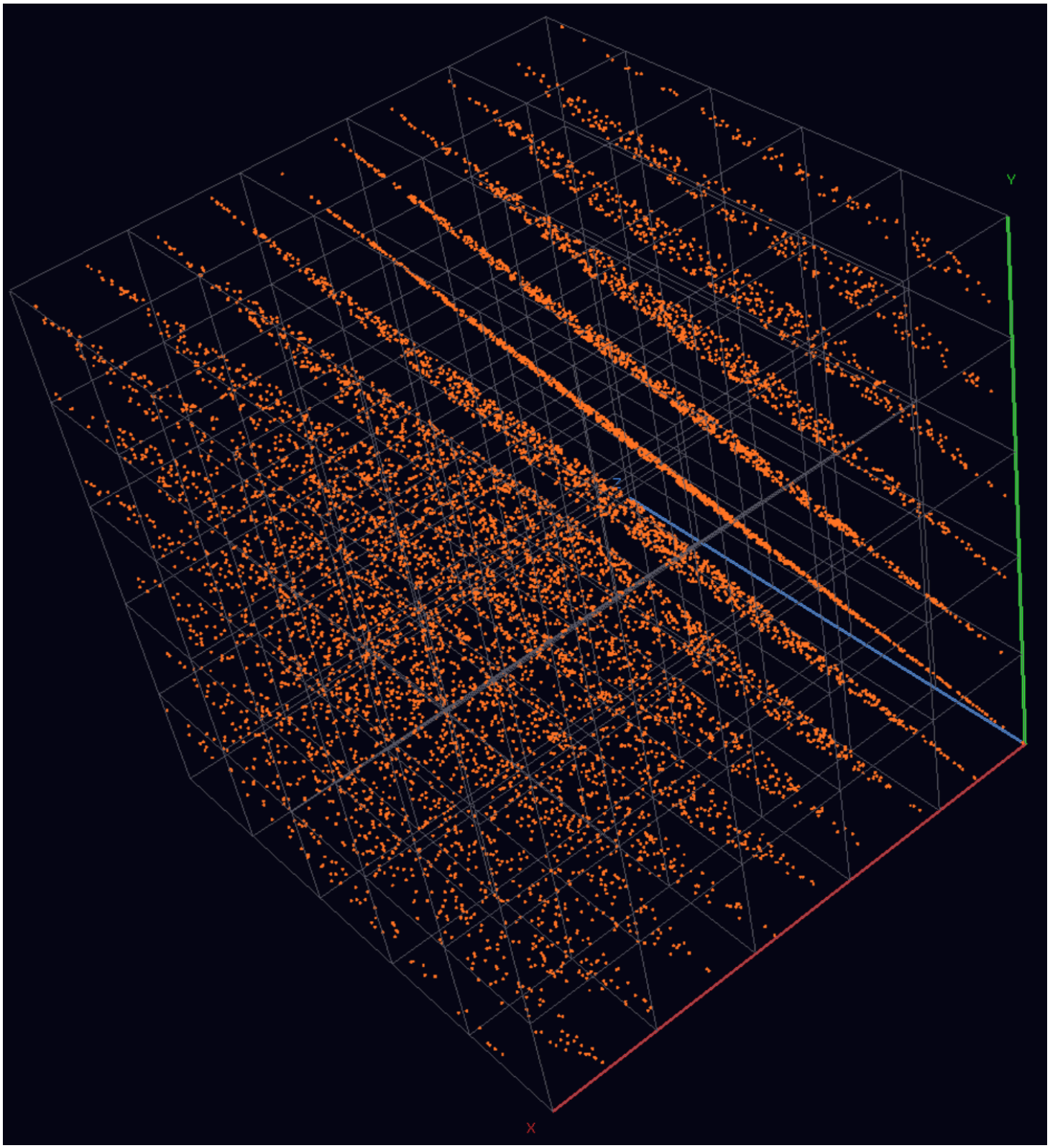


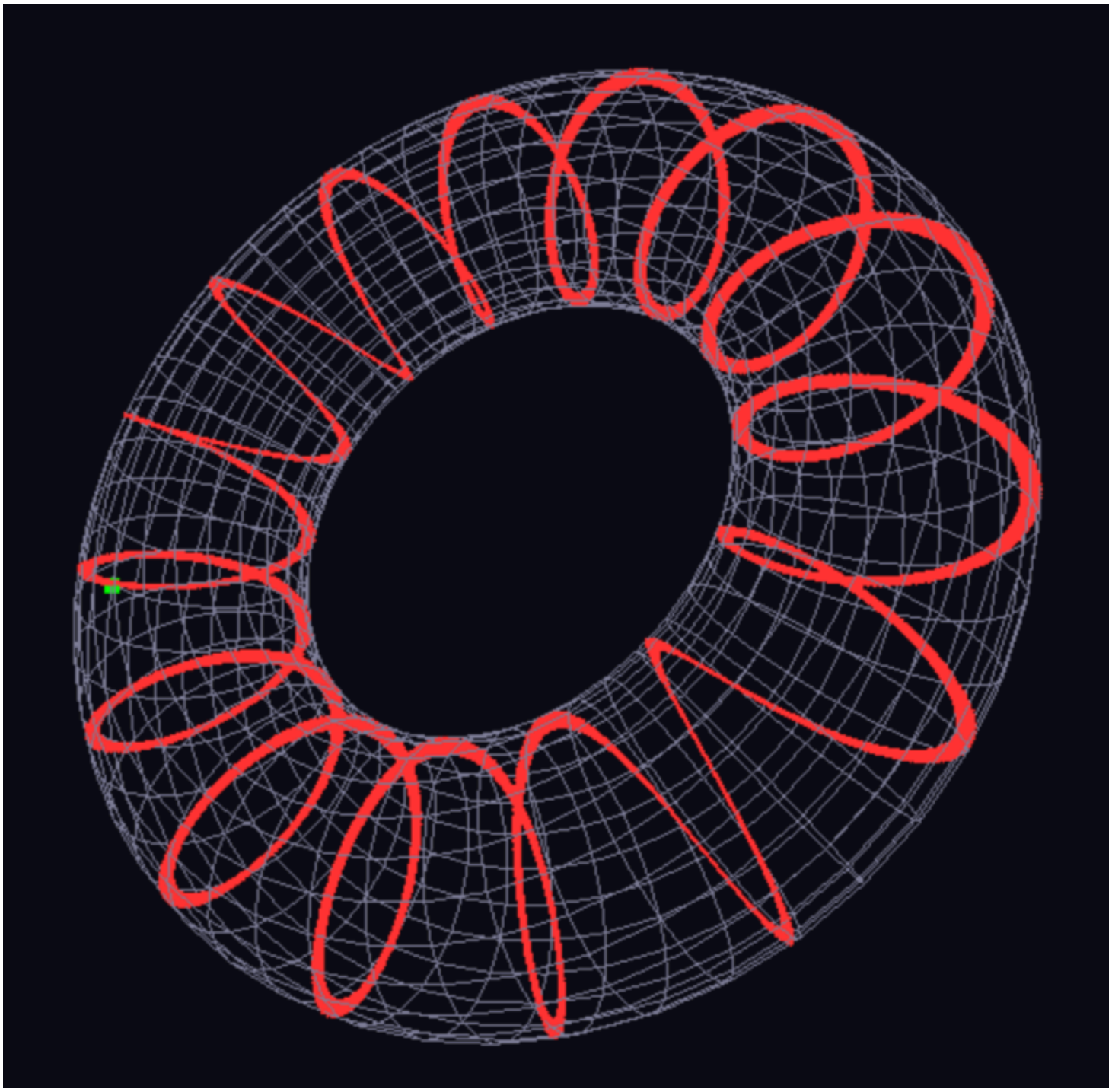
FIGURE 1 A 3D example of the hyperplanes generated by two LCGs with $m = 128$: $a = 37$ (left, $f_3 = 0.545562$) or $a = 29$ (right, $f_3 = 0.833359$). Note the hyperplane structure of the multiplier with a low spectral score

Одна из идей тестирования с ограниченной областью применения предлагается спектральным тестом Д.Кнута. Область применения - линейные конгруэнтные генераторы.



На изображении представлены результаты разложения ГПСЧ RANDU на единичном кубе. Современные методы большой разрядности, включая MWC, не имеют такой явно выраженной решетчатой структуры.

Для визуализации 2D разложения может использоваться представление в виде траекторий и точек на поверхности тора. Тор пердставляет собой идеальную модель, корторая хорошо отображает свойство модульной арифметики - цикличность.



Параметризация поверхности тора в 3D

Это классический «бублик» (torus of revolution).

Параметрические уравнения

$$\begin{cases} x = (R + r\cos\varphi)\cos\theta \\ y = (R + r\cos\varphi)\sin\theta \\ z = r\sin\varphi \end{cases}$$

где

- R — расстояние от центра тора до центра трубки (major radius)
- r — радиус трубки (minor radius), $r < R$
- $\theta \in [0, 2\pi)$ — угол поворота вокруг оси Z (большой круг),
- $\varphi \in [0, 2\pi)$ — угол внутри трубки.

Тор может использоваться для визуализации распределения и траекторий в модульной арифметике.

- см. исходный код модели mwc_torus.c
- см. исходный код модели mwc_cube.c
- Наглядные примеры выделния решетчатых структур в генераторах относятся либо к экстремально низкой разрядности, либо выборке 8 бит из числа.

Решётчатая структура ГПСЧ

- [Lattice Tester Guide]
- (https://github.com/umontreal-simul/LatMRG)

Определения из теории решёток

Решётчатая структура. Аналогом является кристаллическая решётка в $\mathbb{R}^t$, где переход между точками выполняется методом целочисленных трансляций в направлении базисных векторов. Набор линейно независимых векторов, соответствующий трансляциям, составляет *базис* решётки. Для анализа кристаллических решеток может применяться теория групп симметрии, которая оперирует понятием группа трансляций и вращений системы координат. В теории целочисленных решеток не применяется понятие вращение. Не ясно, как анализировать решетчатые структуры для генераторов содержащих нелинейный скрамблер со свойством вращения.

Решётка (Lattice over $\mathbb{Z}$, или $\mathbb{Z}$-решётка) L^t в $\mathbb{R}^t$:

$$L^t = \left\{ \sum_{j=1}^{t} z_j \mathbf{v}_j \;\middle|\; z_j \in \mathbb{Z} \right\},$$

где $\mathbf{v}_1, \ldots, \mathbf{v}_t$ — линейно независимые вектора (составляют базис).

Матрица-генератор решётки (generator matrix) V: матрица, составленная из базисных векторов, i-я строка содержит вектору $\mathbf{v}_i$. Тогда любой вектор решётки записывается, как $\mathbf{v} =$

zV, $z \in \mathbb{Z}^t$.

Определитель решётки (fundamental volume): $\det(L^t) = \det(V)$ — фундаментальный объем не зависит от выбора базиса.

Плотность решётки $\eta^t = 1/\det(L^t)$ — среднее число точек решётки на единицу объёма. Когда $L^t \supseteq \mathbb{Z}^t$, плотность — целое число.

Дуальная решётка L^{*t}:

$$L^{*t} = \{\mathbf{w} \in \mathbb{R}^t \mid \mathbf{w} \cdot \mathbf{v} \in \mathbb{Z} \ \ \forall \mathbf{v} \in L^t\}.$$

Дуальный базис $W = (V^{-1})^t$ удовлетворяет равенству $VW^t = I$.

Целочисленные решётки и m-дуальный базис (m-dual lattice).
Чтобы все координаты были целыми, вводится масштабирование: $\Lambda^t = m_1 L^t \subseteq \mathbb{Z}^t$ и $\Lambda^{*t} = m_2 L^{*t} \subseteq \mathbb{Z}^t$, где $m = m_1 m_2$.
Для m-дуального базиса матрицы V и W связаны соотношением $VW^t = mI$, - это нормированный дуальный базис в целочисленном пространстве.

Проекция решётки L_I — проекция L^t на подмножество координат I. Её плотность η_I делит η^t.

Ключевые проблемы

1. **Lattice Basis Construction**. Для данного множества векторов с целочисленными координатами необходимо найти базис решётки, на которой они располагаются. Базис представляет собой набор линейно независимых базисных векторов, представленный в виде верхне- или нижне-треугольной матрицы.
2. **Нахождение m-дуального базиса** (*m-Dual Lattice Basis*).
   Given a lattice basis (generator matrix V), compute the corresponding m-dual basis W такую, что $VW^t = mI$.
3. **Lattice Basis Reduction** (LBR).
   Given a lattice basis, find another basis whose vectors are as short as possible and/or nearly orthogonal. Существует несколько вариантов редуцирования базиса (LLL, BKZ, Minkowski-reduced и др.).
4. **Shortest Vector Problem** (SVP).
   Find a shortest nonzero vector in the lattice (или в дуальной/m-дуальной решётке), с доказательством, что он действительно кратчайший.

Длина кратчайшего вектора в дуальной решётке — основной праметр спектрального теста для оценки равномерности покрытия объема единичного куба гиперплоскостями.

Типичный рабочий план Lattice Test:

1. Построить базис решетки (треугольный или LLL).
2. При необходимости вычислить m-дуальный базис.
3. Выполнить предредукцию: LLL (быстро) → BKZ (сильнее).
4. Применить BB-алгоритм для нахождения кратчайшего вектора в дуальной решётке.

> В этом месте мы хотим отойти от предложенного плана и выстроить некоторые аналогии из теории Stochastic Time Series, Моделирование стахостических временных последовательностей. Наша задача научиться предсказывать шаг функции и предсказывать начальную точку *SEED*, для поиска паттернов в тесте *Trump aces*.

Основные положения нашей концепции

1. Последвательность проходит статистические тесты и не обладает выраженной структурой решетки в классическом смысле. Однако, возможно определить группу трансляции содержащую бинарную матрицу трансформации (вращение) и линейную трансляцию.
2. Нас интересует выборка из последовательности, которая дает совмещение циклического паттерна или паттерна сложности (с определенным числом ведущих нулей), подбного *Trump Aces*.
3. Предсказание шага выборки одномерного генератора может выполняться в терминах ARIMA (autoregressive integrated moving average) - в модели используется последовательность входных и выходных значений с накоплением результата. Особенность использования метода - совместимость с ARX- структурами обеспечивается за счет переопределения операции умножения и модульной арифметики.
4. Для моделирования генератора в цепи управления шагом мы используем аналогии из теории порт-Гамильтоновых систем и State-Space Model, для систем содержащих *вращение* без диссипации. Вращение описывается матрицей генерации с симплектичесокй структурой или со структурой матрицы-циркулянта. Вращение может выполняться в кольце полиномов (связь с циклотомическими полиномами).
5. Добавление детерминированого квантового шума (Ring-LWE) и компенсация ошибки квантования и дискретизации в структуре Дирака. Структура Дирака возникает, как условие сохранения информации (мощности), как залог обратимости и стабильности модели.

[1] Robert M. Gray. 2005. Toeplitz and circulant matrices: a review. Commun. Inf. Theory 2, 3 (August 2005), 155–239.
https://doi.org/10.1561/0100000006

[2502.05160] A parameter study for LLL and BKZ with application to shortest vector problems

Auto-regressive-модель в модульной арифметике

Эта концепция является *тавтологией* для семейства линейных конгруэнтных генераторов (LCG, MRG и их обобщений). Множественный рекурсивный генератор (Multiple Recursive Generator, MRG) порядка k определяется рекуррентным соотношением

$$x_n \equiv a_1 x_{n-1} + a_2 x_{n-2} + \cdots + a_k x_{n-k} \pmod{m},$$

что представляет собой *авторегрессию порядка* k в кольце $\mathbb{Z}/m\mathbb{Z}$. Поэтому AR-модель (а при отсутствии шума и внешнего управления — чистая AR-модель) здесь естественна и *полностью рабочая* основа для точного предсказания следующих состояний линейного генератора.

Для множественных рекурсивных генераторов естественной и стандартной формой представления является *вектор состояния* и *матрица перехода* (transition matrix, она же companion matrix или generator matrix).

Пусть состояние генератора в момент n задаётся вектором

$$\mathbf{s}_n = \begin{pmatrix} x_n \\ x_{n-1} \\ \vdots \\ x_{n-k+1} \end{pmatrix} \in (\mathbb{Z}/m\mathbb{Z})^k.$$

Тогда следующий шаг записывается в матричной форме

$$\mathbf{s}_{n+1} \equiv A\mathbf{s}_n \pmod{m},$$

где *матрица перехода* A (companion matrix) имеет вид

$$A = \begin{pmatrix} a_1 & a_2 & \cdots & a_{k-1} & a_k \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

Матрицу трансформации A можно рассматривать, как композицию двух частей: матрица перестановок со свойством циркулянта и операция Low-Runk скалярное произведеление векторов (outer product).

В расширенных моделях, с нелинейным скрамблером или «вращением» компонент вектора состояния, вводится дополнительная *матрица перестановок* P (permutation matrix) или более

общая *циркулянтная / блочно-циркулянтная матрица*. В этом случае полный переход может принимать вид

$$\mathbf{s}_{n+1} \equiv P \cdot (A\mathbf{s}_n) \pmod{m}$$

Циркулянтные матрицы особенно удобны, поскольку они диагонализируются с помощью **Number Theoretic Transform (NTT)**, что позволяет эффективно работать во «вращающейся» системе координат и снижать вычислительную сложность.

Такое матричное представление:

- полностью эквивалентно исходному скалярному рекуррентному определению,
- напрямую стыкуется с теорией решёток (базис решётки порождается строками/столбцами матрицы A),
- обобщается на многомерную ARIMA в модулярной арифметике,
- открывает путь к моделированию систем с «генерацией без диссипации» (аналог порт-Гамильтоновых систем) через симплектические или циркулянтные структуры.

Универсальное Разложение матриц в форме Фробениуса

Предпосылки. Не все формы разложения матриц работают в конечных полях $\mathbb{F}_q$ и в GF(2). Всегда существует разложение бинарных матриц в *Rational Canonical Form* (RCF). Разложение RCF тесно связано с факторизацией харакетирстического многочлена матрицы на неприводимые многочлены. Неприводимому многочлену можно сопоставить матрицу-компаньон в форме Фробениуса.

In linear algebra, the Frobenius companion matrix of the monic polynomial

$$p(x) = c_0 + c_1 x + \cdots + c_{n-1} x^{n-1} + x^n$$

is the square matrix defined as

$$C(p) = \begin{bmatrix} 0 & 0 & \dots & 0 & -c_0 \\ 1 & 0 & \dots & 0 & -c_1 \\ 0 & 1 & \dots & 0 & -c_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & -c_{n-1} \end{bmatrix}.$$

Существует несколько эквивалентных представлений матрицы-компаньона. Мы будем использовать *транспонированную форму* матрицы компаньона, которая может оказаться

удобна для представления линейных рекуррентных соотношений и LFSR.

Бинарная матрица-компаньон эквивалентна LFSR, линейному сдвиговому регистру с обратной связью, поскольку сочетает в себе сдвиг и редуцирование по полиному $p(x)$.

Любая квадратная матрица подобна блочно-диагональной матрице, каждый из блоков которой является сопровождающей матрицей. Такая матрица однозначно определяется из исходной квадратной матрицы и называется фробениусовой канонической формой разложения.

Преобразование подобия
Говорят, что матрицы A и B подобны, если существует невырожденная матрица P (матрица подобия) такая, что $B = P^{-1}AP$. Само преобразование матрицы A к виду $B = P^{-1}AP$ называется преобразованием подобия.

- см. Преобразования подобия: итерация Якоби, вращение Гивенса и отражения Хаусхолдера
- группа подобия в конечных полях - унимодулярные матрицы.

Результат разложения RCF — это блочно-диагональная матрица, где каждый блок — сопровождающая матрица неприводимого многочлена - делителя характеристического многочлена.

> Утверждение на уровне того, что любой генератор представленный бинарной матрицей трансформации можно разложить в набор независимых сдвиговых регистров по нередуцируемым полиномам и матрицу преобразования подобия P.

Существует тесная связь между разложением в форме RCF и Жордановой формой, через собственные значения характеристического многочлена.

Сущесвтует представление обыкновенных дифференциальных уравнений (ODE) в форме реккурентных соотношений и представление рекуррентных соотношений в форме сопровождающей матрицы.

In the case of $p(x) = x^n - 1$, when the eigenvalues are the complex roots of unity, the companion matrix and its transpose both reduce to Sylvester's cyclic shift matrix, a circulant matrix.

Свойства RCF в контексте темы выделения структуры решетки:

- Каждый блок в разложении RCF порождает свою собственную решётку в $\mathbb{Z}^d$ (или $GF(2)^d$). Решетка выделяется по диапазонам бит после преобразования подобия.
- Кратчайший вектор (SVP) в дуальной решётке блока напрямую связан с минимальным периодом матрицы команьона.

- Если RCF будет содержать только один большой блок степени 64 с примитивным неприводимым многочленом — решётка «максимально равномерная» (спектральный тест пройден).
- RCF даёт естественный базис, состоящий из независимых циклических компонент.
- Любая кососимметричная матрица-генератор (J), используемая в теории порт-Гамильтоновых систем, в RCF-базисе раскладывается на косые циркулянты.

Каждый блок (матрица-компаньон) RCF порождает независимую циклическую компоненту
RCF представляет матрицу A в виде блочно-диагональной матрицы, где каждый блок — компаньон-матрица $C(p_i)$ неприводимого многочлена $p_i(x)$ степени d_i.
Каждый такой блок эквивалентен отдельному *LFSR* (линейному регистру сдвига с обратной связью) порядка d_i.
Решётка, порождённая этим блоком, выделяется в соответствующем d_i-мерном подпространстве после преобразования подобия $P^{-1}AP$.

Матрица-компаньон порождает генератор с максимальным периодом
Компаньон-матрица $C(p)$ размера $d \times d$ над GF(2) имеет *максимальный порядок* $2^d - 1$ тогда и только тогда, когда характеристический многочлен $p(x)$ является *примитивным* над полем GF(2).

Практическая ценность для тестирования ГПСЧ
RCF позволяет перейти от анализа «чёрного ящика» (стандартные статистические тесты) к *структурному анализу*. Даже если генератор проходит BigCrush/TestU01, наличие нескольких мелких блоков в RCF может объяснить скрытые артефакты в многомерных тестах или в решётчатых структурах, которые проявляются при больших выборках или специфических начальных условиях (`seed`).

Keith R. Matthews "A Rational Canonical Form Algorithm" (1992)

Arne Storjohann. 1998. An O(n3) algorithm for the Frobenius normal form. In Proceedings of the 1998 international symposium on Symbolic and algebraic computation (ISSAC '98). Association for Computing Machinery, New York, NY, USA, 101–105. https://doi.org/10.1145/281508.281570

[2209.04807]

Ключевые концепции алгоритма

1. **Представление статистических данных в форме тензора**: Данные — это тензор размерности $N \times M \times K \times \ldots$ (например, 3D-тензор для событий по категориям). Это

позволяет моделировать многомерные зависимости.

2. **Перестановка (permutation)**: Мы предполагаем использование операций над тензором: перестановки, группировки и нормировки. Рассматриваем возможность проецирования на гиперплоскость. Задача — найти гиперплоскость, на которой данные будут представлены в 2D и обладать контрастным распределением плотности.
3. **Группировка блоков (folding)**: Разбиваем тензор на блоки и агрегируем их (сумма, среднее, максимум и т.д.), имитируя "складывание салфетки". Это помогает выявить скрытые паттерны.
4. **Выявление паттернов**: После каждой операции проверяем распределение в полученной матрице на отклонение от нормальности. Используем:
   - Статистические тесты (например, тест Шапиро-Уилка для нормальности; тест Колмогорова-Смирнова).
   - Визуализацию (гистограммы, 2D-heatmaps) для выявления "узоров"/"водяных знаков".
   - Метрики отклонения: kurtosis (эксцесс), skewness (асимметрия), при сравнении с нормальным распределением.
   - Спектральный анализ: (sspectral_Fourier1/3 или кастомный FFT), графический поиск гиперплоскостей: периодические полосы в 'салфетках'.
5. **Итеративный поиск**: Перебираем комбинации перестановок и группировок, чтобы найти конфигурацию с наибольшим отклонением. Для оптимизации можно использовать стохастический градиентный спуск.

Базовые идеи, которые работают в любых конечных полях (коммутативных кольцах с единицей).

1. Разложение матрицы в форме *Rational Canonical Form*(RCF) всегда существует и единственно для любых полей
2. Построение матрицы $(xI - A)$ и её *Smith normal form* (SNF) - приведение к диагональному виду, подобно характеристическому полиному.
3. Построение циклического подпространства Крылова (Krylov subspace) из ряда $\{v, Av, A^2v, ...A^{k-1}v\}$, где A - генераторная матрица. Полученные вектора используются, как базис для построения U - матрицы подобия.
4. Ортогонализация базиса. Нужна подобная идея, в конечных полях, где операция скалярного произведения не определена. Подходят методы подобные Гауссу (Gauss elimination), которые опираются только на арифметические операции поля. Используется приведение матрицы к треугольному виду и к форме Хессенберга.
5. Циклические шаблоны и периоды в тесте Trump Aces.
   После преобразования подобия $U^{-1}AU$ матрица становится блочно-диагональной.
   Каждый блок (клетка Жордана или матрица-компаньон) порождает свой повторяющийся паттерн.
   Период повтора паттерна определяется порядком матрицы-компаньона (или порядком

минимального полинома в кольце). Циклические шаблоны в тесте Trump Aces следует искать после приведения матрицы к RCF, каждая клетка блочно-диагональной матрицы даёт независимый период повтора. "Тузы следуют парами" - этот тезис раскрывается в полной мере, при наличии нулей, в младшем(старшем) слове, после применения операции сдвига, редуцирования в кольце не происходит, - это особенно четко видно на примере LFSR регистров, но работает точно также и в кольце $R_q = \mathbb{Z}_q/p(x)$.

6. Связь с циклотомическими полиномами. При использовании циклотомических полиномов можно установить связь между:
кольцами $\mathbb{Z}_q[x]/(\Phi_n(x))$ и циклотомическим полем $\mathbb{Q}(\zeta_n)$, где ζ_n — примитивный n-й корень из единцы. Такие поля используются в пост-квантовой криптографии на решетках (Ring-LWE) и ZKP. Вычисления в циклотомических полях можно производить с использованием FFT (и его аналога NTT, в конечных полях) в качестве матрицы подобия. Вместе это дает основу построения примитивов для пост-квантовой криптографии.

- Правильная связь: $\Phi_n(x)$ неприводим над $\mathbb{Q}$, поэтому $\mathbb{Q}[x]/(\Phi_n(x)) \cong \mathbb{Q}(\zeta_n)$.

> Я помню из обсуждения, многократное применение матицы трансформации, в подпространстве Крылова, работает как аналог принципа сжимающих отображений. Путем возведения в степень (exponentiation, power iteration) можно получить матрицу содержащую собственные числа на главной диагонали. В контексте обсуждения матрица будет иметь свойство генератора (со своим периодом) и содержать в себе матрицу диссипации, которая задавит все компоненты кроме диагональных.
> В линейной алгебре присутствует концепция Power Iteration и Orthogonal Iteration [Golub & van Loan] со схожей идеей: выстроить рекурсивную последовательность ортогональных преобразований (или преобразований подобия) чтобы получить блочно-диагональную стрктуру матрицы. Концепция не чувствительности собственных значений к преобразованиям подобия так же присутствует.

Сжимающее отображение — отображение метрического пространства в себя, уменьшающее расстояние между любыми точками в некотором сильном смысле.

Определение. Пусть на метрическом пространстве $(\mathbb{M}, d)$ определено отображение $A : \mathbb{M} \to \mathbb{M}$. Оно называется сжимающим на $\mathbb{M}$, если существует такое неотрицательное число $0 \leq \alpha < 1$, что для любых двух точек $x, y \in \mathbb{M}$ выполняется неравенство $d(Ax, Ay) \leqslant \alpha\, d(x, y)$.
Число $\alpha < 1$ называют коэффициентом сжатия.

> Для использования этого определения необходимо ввести понятие дистанция через понятие *норма* метрического пространства. Нужно другое определение подобное сохранению спектра и мощности для матриц генерации, аналогично разделению матрицы

> на симелектическую и диссипативную. В порт-гамильтоновых системах при отсустии вншегео воздействия на генерацию работает симплектическая матрица. В теории PHS структура Дирака обеспечивает сохранение мощности.

https://arxiv.org/pdf/2307.12261

Собственные значения и инвариантные подпространства

Собственные числа матрицы $A \in \mathbb{C}^{n \times n}$ are the n roots являются корнями характеристического многочлена $p(z) = det(zI - A)$. Множество корней назывется *спектром матрицы* A и обоазначается $\lambda(A) = \{z : det(zI - A) = 0\}$

Теорема. Спектр матрицы A совпадает со спектром матрицы $P^\top AP$ при произвольной ортогональной матрице P.

Теорема. Собственные числа вещественной симметричной матрицы - вещественные числа.

Теорема. Собственные числа косо-симметричной вещественной матрицы - мнимые числа и возможно ноль.

Теорема. Собственные числа вещественной ортогональной матрицы все равны 1 по абсолютной величине (модулю).

Путем применения преобразований подбия (similarity transformations) возможно привести матрицу к одному из канонических представлений. Канонические представления отличаются способом представления собственных значений и тем как они отображают инвариантные подпространства.

- Разложение Шура
- Power Iteration
- Orthogonal Iteration
- QR-алгоритм для поиска собственных значений

Матрица-генератор в подпространстве Крылова

Многократное применение матрицы трансформации A к начальному вектору v в циклическом подпространстве Крылова

$$\mathcal{K}_k(A, v) = \mathrm{span}\{v,\ Av,\ A^2v, \ldots,\ A^{k-1}v\}$$

выполняет роль *спектрального фильтра*.

Матрица A выступает как *генератор* мультипликативной группы, действующий на конечномерном векторном пространстве над полем.

- Размерность и Ивариантность подпространства
- Метод Крылова для вычисления коэффициентов характеристического многочлена
- Теорема Гамильтона-Кэли

Ключевые свойства в конечных полях:

- Инвариантом здесь является *подпространство*, соответствующее каждому неприводимому множителю минимального полинома.
- Значение "ноль" в клетке является неподвижной точкой. Генератор теряет свойства при таких начальных значениях.
- Собственные числа (корни характеристического полинома) в комплексном замыкании *ходят парами* (сопряжённые пары), что проявляется в структуре RCF как парность блоков с сопряжёнными (обратными) полиномами.
- Понятие непрерывности заменяется *полнотой* генератора: матрица A порождает мультипликативную группу с максимальным периодом на каждом циклическом подпространстве.
- Понятие орбит. Максимальный период мультипликативной группы может быть не полным. В этом случае пространство разбивается на несколько непересекающихся орбит. Каждая орбита будет содержать свой набор циклических паттернов.
- Дуальный базис и m-дуальная генераторная матрица $VW^t = mI$ из теории решеток. Идея происходит из обращения целочисленных матриц, где вместо деления вводится масштабирование, что в численных методах заменяет рациональные числа и позволяет сохнить понятие скалярного произведения.
- Изомофризм конечных полей. Гомоморфизм. Надо иметь ввиду, что два конечных поля с одинаковым числом состояний изоморфны. Изоморфизм будет определяться выбором генератора поля, начальной точкой. Изоморфизм дается прямой и обратной матрицей аффинного преобразования. Изоморфизм связывает два поля, принимая генератор поля за единицу - начало отсчета. Точно так же можно построить гомоморфизм используя два разных генератора, две разные начальные точки в одном поле.

Анализ скрамблеров через Rational Canonical Form

Он связывает *универсальную параметризуемую функцию-скрамблер* с RCF, подпространством Крылова, полиномиальным представлением и тестом Trump Aces.

Скрамблеры в полиномиальном представлении

В криптографических хеш-функциях SHA-2 и SHA-3/Keccak для перемешивания слов состояния используются *скрамблеры* — биективные преобразования над машинным словом (32/64 бита) и циклические перестановки над вектором состояния.

Мы различаем два типа скрамблеров:

- **Линейные (по GF(2))** — Sigma0, Sigma1, Sum0, Sum1 в SHA-2 и θ, ρ, π в Keccak.
- **Нелинейные** — Maj, Ch в SHA-2 и χ в Keccak.

Линейный скрамблер как полиномиальное умножение

Любой линейный скрамблер вида

$$x \;\mapsto\; \mathrm{ROTR}(x,a) \oplus \mathrm{ROTR}(x,b) \oplus \mathrm{ROTR}(x,c)$$

с произвольным набором вращений, эквивалентен *полиномиальному умножению* в кольце

$$R = \mathbb{F}_2[x]/(x^w+1),$$

где w — разрядность слова (32 или 64). Набору сдвигов можно сопоставить полином.

Матрица этого преобразования в стандартном базисе — *циркулянтная*.

Операция умножения на полиномов реализуется, как умножение «без переносов» (carry-less multiplication) с последующим редуцированием по модулю x^w+1.

Универсальная параметризуемая функция-скрамблер (для 32/64-бит):

$$S_p(x) = p(x)\cdot x \bmod (x^w+1), \quad p(x)\in\mathbb{F}_2[x],\ \deg p < w.$$

Линейный скрамблер обратим через обратный полиномом, поскольку $\gcd(p(x), x^w+1) = 1$ (что выполнено для всех Sigma/Sum в SHA-2), полином $p(x)$ обратим в кольце.

Разложение скрамблера на клетки Жордана/матрицы-компаньоны (RCF)
Матрица линейного скрамблера S_p (размер $w \times w$ над $\mathbb{F}_2$) всегда может быть приведена к *Rational Canonical Form* (RCF) — блочно-диагональному виду, где каждый блок — *матрица-компаньон* неприводимого множителя характеристического полинома матрицы.

Алгоритм перехода к RCF:

1. Строим характеристический полином матрицы S_p.
2. Факторизуем его на неприводимые полиномы над $\mathbb{F}_2$.
3. Для каждого неприводимого множителя $f_i(x)$ степени d_i строим матрицу-компаньон $C(f_i)$.
4. Находим матрицу подобия U через подпространства Крылова такую, что

$$U^{-1} S_p U = \operatorname{diag}\big(C(f_1),\ C(f_2),\ \dots\big).$$

5. Используем разбиение вектора состояния на диапазоны бит при выявлении независимых циклических паттернов в каждой клетке.

Синтез функций соответствующих разложению

Одна клетка блочно-диагональной матрицы в RCF содержит матрицу компаньон по полиному p(x).
В двоичных числах структура матрицы компаньона - эквивалентна LFSR, сдвиговому регистру с обратной связью. На каждом шаге происходит сдвиг и редуцирование по неприводимому полиному.

В конечном поле по модулю простого числа: $\mathcal{R}_q = \mathbb{Z}_q[x]/p(x)$. Операция состоит из циклической перестановки элементов вектора и свертки (outer product)

$$\begin{aligned} v_1^{(k+1)} &= -v_n^{(k)} \cdot c_1 \\ v_i^{(k+1)} &= v_{i+1}^{(k)} - v_n^{(k)} \cdot c_i, \quad 1 < i \le n \end{aligned}$$

Или в транспонированной форме

$$\begin{aligned} v_i^{(k+1)} &= v_{i-1}^{(k)}, \quad 1 \le i < n \\ v_n^{(k+1)} &= \sum_{j=1}^{n} v_j^{(k)} \cdot c_j \end{aligned}$$

Две формы представления: прямая и транспонированная - подобны. В прямой форме используются три векторные операции: SHUFFLE, BROADCAST, vMUL - поэлементное умножение на вектор констант.
В транспонированной форме действие клетки аналогично операции AR- авторегрессии с задержкой или цифровому фильтру с бесконечной импульсной характеристикой. Вопрос: можно ли представить действие клетки канонического разложения в форме ARIMA?

В структуру вычислительного графа надо добавить понятие входной и выходной порт и принцип повышения и понижения частоты дискретизации. Матрица подобия - аффинное преобразование применяентся к входному порту. Для начала мы должны рассмотреть цикл относительно принципа работы генератора случайный чисел: Сначала выполняется загрузка `seed` с пересчетом на вектор внутреннего состояния, затем - пошаговая генерация с выводом значений через матричное умножение и скрамблер.

Задачи:

- представить функцию CRC в конечном поле.
- представить стохастическую SSM, понятие *observable* и *controllable*.
- представить мультифазный цифровой фильтр в канонической форме;
- хеш-функция sponge-queeze в канонической форме
- каноническое разложение кода Рида-Соломона в изоморфном представлении поля.

The concept of observability was introduced by the Hungarian-American engineer Rudolf E. Kálmán for linear dynamic systems.

Понятия Observability и Controllability введены Р. Е. Калманом для линейных динамических систем, и применяется для State-Space Model[1]. Математически критерий Управляемости (в теории управления) определяются через соотношение матрицы B и A, путем построения подпространства Крылова, где B может снизить размерность пространства (когда ранг матриц $\{A^k B\}$ оказывается ниже рагнга матриц $\{A^k\}$). В этом случае система не полностью управляема. При каноническом разложении часть вектора состояния не будет зависеть от входных данных.

[1] R.E. Kalman, MATHEMATICAL DESCRIPTION OF LINEAR DYNAMICAL SYSTEMS, 1963

В этом контексте мы рассматриваем часть состояния, как не контролируемое. Стохастическая компонента вектора состояния - это та, которую мы не можем контроллировать через внешний порт. Таким образом мы говорим про два генератора, выход стохастической компоеннты используется для генерации состояния. В теме генераторы ГПСЧ такая конструкция будет соответствовать комбинированному генератору. Понятие порт внешнего управления не применяется к ГПСЧ, только к хэш-функции, в которой присуствует подмешивание данных с порта $u(t)$ через матрицу B.